



Università degli Studi di Torino

Corso di Laurea in Informatica

**Traduzione dell'Input in Linguaggio Naturale Controllato
per Chatbot esperto di Automi a Stati Finiti mediante
Large Language Model**

Primo Relatore

Prof. Luca Anselma

Secondo Relatore

Prof. Alessandro Mazzei

Correlatore

Dott. Pier Felice Balestrucci

Candidato

Davide Candela

Matricola: 976529

Anno Accademico 2023/2024

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Indice

1	Introduzione	1
1.1	Obiettivi della tesi e il chatbot NoVAGraphS	2
1.2	Struttura del documento	3
2	Stato dell'arte	5
2.1	Automi a stati finiti	5
2.2	Sistemi di dialogo e chatbot	7
2.3	AIML	9
2.4	Reti neurali e Large Language Model	10
2.5	Linguaggio naturale controllato	14
3	Descrizione e implementazione dell'architettura di traduzione	15
3.1	Descrizione generale del sistema	15
3.2	Gestione dell'input dell'utente con il server Flask mediante ngrok	19
3.3	Verifica della pertinenza mediante vector store	20
3.4	Ricerca di esempi pertinenti (few-shot) mediante vector store	22
3.5	Traduzione in CNL mediante LLM	24
3.6	Output finale	27
4	Valutazione del sistema e analisi dei risultati	28
4.1	Valutazione delle performance del sistema	28
4.2	Analisi dei risultati della valutazione	32
5	Conclusioni, risultati ottenuti e possibili sviluppi futuri	39
	Bibliografia	41

Capitolo 1

Introduzione

L'elaborazione del linguaggio naturale (*Natural Language Processing*¹) ha visto una crescita esponenziale negli ultimi anni, con l'aumento dell'uso di *chatbot*² e assistenti virtuali. Lo scopo della NLP è quello di sviluppare modelli e algoritmi che siano in grado di *comprendere*, interpretare e fornire una risposta in linguaggio naturale, ossia il normale linguaggio di uso quotidiano degli esseri umani. Tuttavia, la comprensione e l'interpretazione del linguaggio naturale da parte delle macchine sono una sfida significativa, specialmente quando si tratta di tradurre input complessi. La comprensione del linguaggio naturale è spesso considerata un problema IA-completo³, poiché si ritiene che per riconoscere e interpretare il linguaggio sia necessaria una conoscenza approfondita del mondo. Per questa ragione, non si può attualmente parlare di una vera e propria comprensione da parte delle macchine. Tutti i modelli di elaborazione del linguaggio naturale attualmente, pur operando attraverso meccanismi diversi, condividono lo stesso tipo di limitazione legati proprio al concetto di comprensione. Ad esempio i chatbot generativi basati su modelli di linguaggio di grandi dimensioni (*Large Language Model*⁴) non possiedono una comprensione semantica del contenuto, ma predicono la parola successiva, basandosi su correlazioni statistiche, derivate da grandi quantità di dati testuali. Allo stesso modo i chatbot con risposte predefinite o basati su regole (*rule-based chatbot*) si limitano a selezionare risposte fisse sulla base di regole o pattern specifici, ma non possono gestire input complessi o ambigui al di fuori degli schemi preimpostati. In entrambi i casi, la capacità di comprendere il significato delle conversazioni è assente, il che rappresenta uno dei maggiori problemi dell'elaborazione del linguaggio naturale. Per ulteriori approfondimenti fare riferimento alla Sezione 2.2 e alla Sezione 2.4.

In questo contesto, il progetto presentato in questa tesi, si focalizza sull'ottimizzazione delle interazioni tra gli utenti e tra un chatbot scritto in *AIML*⁵, tramite l'uso di un Large Language

¹Il Natural Language Processing si riferisce a un insieme di algoritmi di intelligenza artificiale progettati per analizzare, rappresentare e comprendere il linguaggio umano [8].

²Un chatbot è un programma informatico progettato per imitare e gestire conversazioni umane, sia testuali che vocali, permettendo agli utenti di interagire con dispositivi digitali come se dialogassero con una persona reale [13].

³Il termine "AI-completo" indica problemi di intelligenza artificiale che richiedono capacità cognitive simili a quelle umane per essere risolti. Si tratta di problemi considerati complessi, come la comprensione del linguaggio naturale [15].

⁴Un Large Language Model (LLM) è una tipologia di modello linguistico capace di comprendere e generare testo in un'ampia gamma di contesti linguistici [24].

⁵AIML (Artificial Intelligence Markup Language) è un linguaggio di markup simile a XML, creato per stabilire

Model e tecniche di traduzione del linguaggio naturale in linguaggio naturale controllato (CNL⁶).

1.1 Obiettivi della tesi e il chatbot NoVAGraphS

L'obiettivo principale di questa tesi è quello di sviluppare un sistema di traduzione per un chatbot basato su AIML. Si vuole infatti trasformare l'input dell'utente, espresso in linguaggio naturale, in un formato che possa essere compreso facilmente ed efficacemente dal motore AIML. Infatti, riducendo le ambiguità presenti nell'input originale, si migliora l'accuratezza delle risposte del chatbot. La sfida principale da affrontare è quella di costruire un'architettura e un sistema, in grado di gestire le complessità e le ambiguità proprie del linguaggio naturale e, in questo caso di studio, dell'inglese. Lo scopo finale è infatti quello di migliorare la qualità e la precisione delle interazioni fra l'utente e il chatbot.

Un altro aspetto fondamentale della tesi è la valutazione della precisione e dell'efficacia del sistema. In particolare è stata misurata la rilevanza e la precisione delle risposte generate e, per fare ciò, sono stati raccolti e analizzati feedback dagli utenti tramite un'interfaccia grafica.

Questa tesi si basa su un chatbot, denominato NoVAGraphS, realizzato presso il dipartimento di informatica dell'Università di Torino, dal dottorando Pier Felice Balestrucci. Il chatbot in questione è stato pensato come un strumento utile nel campo dell'accessibilità, in particolare per quanto riguarda la comprensione degli automi a stati finiti deterministici da parte di persone non vedenti. Fare riferimento alla Sezione 2.1 per ulteriori informazioni riguardanti gli automi a stati finiti.

Gli automi a stati finiti (*Finite State Automaton*) sono tradizionalmente rappresentati in forma grafica o tabellare. Tuttavia, queste rappresentazioni non sono sempre facilmente interpretabili da persone con disabilità visiva. Per questo motivo, il chatbot NoVAGraphS è stato sviluppato con l'obiettivo di colmare questa lacuna, offrendo una descrizione verbale di uno specifico FSA che sia accessibile e comprensibile. Il sistema è realizzato utilizzando il linguaggio AIML, che permette di creare risposte basate su pattern predefiniti per rispondere alle domande degli utenti riguardanti la struttura e il funzionamento dell'automa a stati finiti. L'architettura del chatbot NoVAGraphS è mostrata in Figura 1.1, dove si possono osservare i vari componenti che costituiscono il sistema.

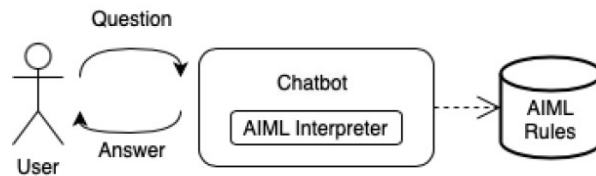


Figura 1.1. Architettura del chatbot NoVAGraphS (Fonte: [2]).

le regole di conversazione che un chatbot deve rispettare durante le interazioni [21].

⁶Il Controlled Natural Language è un tipo di linguaggio naturale strutturato con l'obiettivo di ridurre le ambiguità e facilitare l'elaborazione automatica.

Un esempio di regola AIML utilizzata dal chatbot è illustrato in Figura 1.2.

```
<category>
  <pattern>HOW MANY STATES *</pattern>
  <template>
    There are a total of 3 states.
    q0, q1, and q2. q0 is both initial and final state.
  </template>
</category>
```

Figura 1.2. Esempio di regola AIML (Fonte: [2]).

Inoltre, il sistema fornisce un'interfaccia, rappresentata in Figura 1.3, che facilita e rende accessibile la comprensione delle informazioni riguardanti gli automi a stati finiti [19].

Question:

Send

There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both initial and final state.

History of Q&A

#	Question	Answer
3	how many states?	There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both initial and final state.
2	is there a pattern?	There is only one arc starting from each state. The first three in clockwise direction are marked with 1, the remaining two with 0.
1	how many arcs	There are a total of 5 arcs. Try asking me if there is a particular pattern among them.

Figura 1.3. Interfaccia del chatbot NoVAGraphS (Fonte: [2]).

1.2 Struttura del documento

Il presente documento è suddiviso in cinque capitoli che guidano il lettore attraverso l'intero percorso della ricerca e implementazione sviluppata in questa tesi. Il secondo capitolo si concentra sullo stato dell'arte, esplorando i principali concetti teorici alla base del progetto, tra cui gli automi a stati finiti, i sistemi di dialogo, i modelli di linguaggio di grandi dimensioni (LLM), il linguaggio AIML e il linguaggio naturale controllato. Il terzo capitolo entra nel vivo della descrizione e implementazione dell'architettura di traduzione proposta. Vengono illustrate le componenti tecniche del sistema, l'uso del vector store e dei modelli di embedding per verificare la pertinenza dell'input, nonché le tecniche di few-shot prompting per migliorare la traduzione. Il quarto capitolo è dedicato alla valutazione delle performance del sistema, sia tramite feedback degli utenti sia mediante un'analisi esperta, fornendo un quadro dettagliato dei risultati ottenuti. Infine, il

quinto capitolo riassume i principali risultati raggiunti, riflette sui limiti del sistema sviluppato e propone possibili direzioni per futuri sviluppi.

Capitolo 2

Stato dell'arte

2.1 Automi a stati finiti

Gli automi a stati finiti (o macchine a stati finiti) sono modelli matematici di computazione utilizzati per rappresentare sistemi che hanno un numero finito di stati e che cambiano stato in base a determinati input. Formalmente, un automa a stati finiti può essere così definito:

$$A = (S, \Sigma, \delta, s_0, F)$$

dove:

- S è l'insieme finito di stati, che rappresentano le diverse situazioni in cui il sistema può trovarsi;
- Σ è l'alfabeto finito di simboli di input, che rappresenta i possibili input che il sistema può ricevere;
- $\delta : S \times \Sigma \rightarrow S$ è la funzione di transizione, che specifica come l'automa passa da uno stato all'altro in base ad un input;
- $s_0 \in S$ è lo stato iniziale, ossia lo stato in cui l'automa si trova all'inizio del processo;
- $F \subseteq S$ è l'insieme di stati finali, stati in cui l'automa può terminare dopo una sequenza di input.

Gli automi a stati finiti leggono stringhe di simboli di input appartenenti all'alfabeto e, in base alla sequenza ricevuta, eseguono transizioni da uno stato all'altro. Alla fine della sequenza, l'automa si troverà in uno stato finale o non finale, determinando se la stringa viene accettata o meno [1].

Ci sono due principali tipi di automi a stati finiti che si distinguono per il modo in cui gestiscono le transizioni: gli automi deterministici (DFA) e non deterministici (NFA).

Automa a stati finiti deterministico (DFA)

Un automa a stati finiti deterministico (*Deterministic Finite Automaton*, *DFA*) ha una funzione di transizione δ che mappa ogni coppia di stato e simbolo di input ad un unico stato. In altre

parole, per ogni stato e per ogni simbolo dell'alfabeto di input, esiste una e una sola transizione possibile. Questa caratteristica rende il comportamento dell'automa completamente prevedibile e deterministico [1].

La Figura 2.1 mostra un esempio di tabella di transizione per un DFA, mentre la Figura 2.2 rappresenta graficamente le transizioni tra stati.

δ	a	b
q_0	q_0	q_1
q_1	q_2	q_2
q_2	q_2	q_2

Figura 2.1. Tabella di transizione di un DFA (Fonte: [1]).

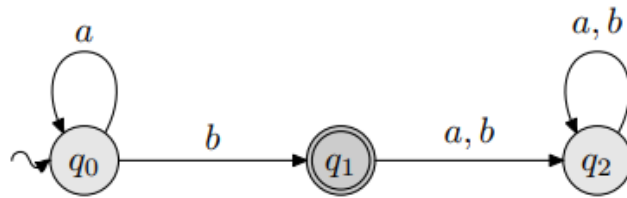


Figura 2.2. Rappresentazione di un DFA (Fonte: [1]).

Automa a stati finiti non deterministico (NFA)

Un automa a stati finiti non deterministico (*Nondeterministic Finite Automaton*, NFA) differisce dal DFA perché la funzione di transizione δ può mappare una coppia di stato e simbolo di input ad un insieme di stati. Ciò significa che, per una stessa coppia di stato e simbolo, possono esistere più transizioni possibili, oppure nessuna. Inoltre, un NFA può includere transizioni *epsilon* (ϵ), che permettono di passare da uno stato ad un altro senza consumare alcun input [1].

La Figura 2.3 presenta un esempio di tabella di transizione per un NFA, mentre la Figura 2.4 mostra una rappresentazione grafica di un NFA.

δ_N	a	b
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2, q_3\}$	$\{q_3\}$
q_2	$\{q_3\}$	$\emptyset$
q_3	$\emptyset$	$\emptyset$

Figura 2.3. Tabella di transizione di un NFA (Fonte: [1]).

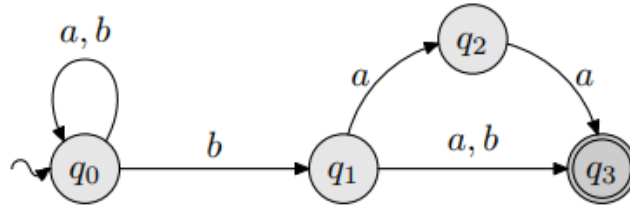


Figura 2.4. Rappresentazione di un NFA (Fonte: [1]).

2.2 Sistemi di dialogo e chatbot

I sistemi di dialogo (*dialogue systems*) rappresentano una delle applicazioni più importanti e complesse dell'intelligenza artificiale e dell'elaborazione del linguaggio naturale. Questi sistemi hanno lo scopo di interagire con gli esseri umani attraverso il linguaggio naturale, simulando conversazioni sia per scopi pratici che per intrattenimento. Esistono due principali categorie di sistemi di dialogo: i sistemi orientati ai compiti e i chatbot [13].

Sistemi orientati ai compiti

I sistemi orientati ai compiti (*task-oriented dialogue agents*), come quelli integrati negli assistenti digitali (*Siri, Alexa, Google Assistant*), sono progettati per aiutare gli utenti a completare attività specifiche. Esempi di compiti che questi assistenti sono in grado di svolgere includono la ricerca di informazioni, la prenotazione di un volo o il controllo di dispositivi domestici. Questi sistemi utilizzano un'architettura basata su *frame*. Ogni frame rappresenta una specifica attività con vari *slot* da riempire con informazioni fornite dall'utente [13].

L'esempio più emblematico di sistema di dialogo basato su frame è l'architettura *GUS* (*Genial Understanter System*), sviluppata negli anni '70 per condurre conversazioni orientate a compiti specifici, come la pianificazione di viaggi.

```

GUS: Hello. My name is GUS. I can help you plan a simple trip by air. Where do you
      want to go ?
Client: I want to go to San Diego on May 28
GUS: What time do you want to leave?
Client: I must be in San Diego before 10 am
GUS: Would you like P.S.A. flight 102 that arrives at 9.15 am ?
Client: What is the next flight ?
GUS: Air California flight 310 that leaves at 8.30am and arrives at 10.00am
Client: I'll take the first one
GUS: In what name should I make a reservation, please?
Client: Dan Bobrow
GUS: I have confirmed the following flight: P.S.A. flight 102 on Wednesday May 28
      from San Jose to San Diego that leaves at 7.30 am and arrives at 9.15 am.
GUS: What date do you want to return on ?
Client: On Friday in the evening.
GUS: Would you like the flight that leaves at 7.45 pm ?
Client: That's fine.
GUS: I have confirmed the following flight: P.S.A. flight 307 on Friday May 30 from
      San Diego to San Jose that leaves at 7.45 pm and arrives at 9.30 pm Thank you
      for calling. Goodbye
  
```

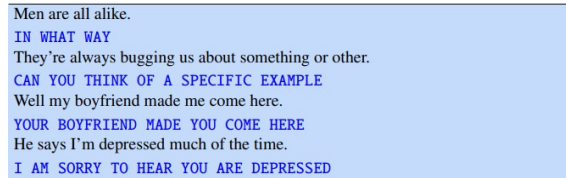
Figura 2.5. Conversazione tra un utente e GUS, un chatbot basato su frame (Fonte: [3]).

Come si può notare in Figura 2.5, il sistema guida la conversazione raccogliendo informazioni dall'utente tramite domande mirate e riempiendo gli slot all'interno di frame strutturati che

rappresentano le informazioni rilevanti per il compito. Ad esempio, in un contesto di prenotazione di voli, i frame contengono slot per dati come la città di partenza, la destinazione e l'orario del volo. Dunque lo scopo del sistema è quello di riempire questi slot ponendo una serie di domande all'utente, per poi eseguire l'azione richiesta, come la ricerca di un volo disponibile [3].

Chatbot

I chatbot, a differenza dei sistemi orientati ai compiti, sono progettati per sostenere conversazioni più lunghe e meno strutturate. Lo scopo principale dei chatbot è spesso quello di intrattenere e rendere più naturali i sistemi orientati ai compiti. Storicamente, uno dei primi chatbot è stato *ELIZA*, sviluppato negli anni '60, che simulava un terapeuta utilizzando semplici pattern di trasformazione del linguaggio. *ELIZA* funzionava attraverso regole predefinite che riflettevano le affermazioni dell'utente, creando l'illusione di una conversazione umana [28].



```
Men are all alike.  
IN WHAT WAY  
They're always bugging us about something or other.  
CAN YOU THINK OF A SPECIFIC EXAMPLE  
Well my boyfriend made me come here.  
YOUR BOYFRIEND MADE YOU COME HERE  
He says I'm depressed much of the time.  
I AM SORRY TO HEAR YOU ARE DEPRESSED
```

Figura 2.6. Conversazione tra un utente e *ELIZA*, uno dei primi chatbot sviluppati (Fonte: [28]).

I chatbot moderni possono essere classificati in tre principali categorie [13]:

- **Chatbot basati su regole (*rule-based chatbots*):** Come *ELIZA*, questi chatbot seguono pattern predefiniti e regole che determinano le risposte basate sugli input dell'utente. Un esempio tipico è un *FAQ bot* utilizzato nell'assistenza clienti, che associa input specifici a risposte corrispondenti. Questi chatbot si basano su *pattern matching* e non generano nuove risposte, ma rispondono sempre secondo le regole impostate. Dunque i chatbot rule-based sono adatti a compiti semplici e ripetitivi.
- **Chatbot basati su corpora (*corpus-based chatbots*):** Utilizzano tecniche di *information retrieval* per cercare risposte appropriate in grandi dataset di conversazioni umane. Invece di generare risposte nuove, questi chatbot selezionano la risposta più simile a quella attesa dall'utente, confrontando il suo input con le conversazioni presenti nel corpus. Questo tipo di chatbot è più flessibile rispetto ai chatbot basati su regole, ma non è in grado di generare risposte dinamiche come i modelli basati su reti neurali.
- **Chatbot con architetture avanzate:** I chatbot con architetture avanzate utilizzano reti neurali profonde basate su modelli *encoder-decoder*, come quelli impiegati nei Large Language Model (LLM). Per ulteriori dettagli sui Large Language Model, vedere la Sezione 2.4. Questi chatbot non recuperano semplicemente risposte preesistenti, ma generano risposte nuove basate sul contesto della conversazione. Ad esempio, *ChatGPT*, basato su architettura *transformer*, è un esempio di LLM che può gestire conversazioni complesse, generare risposte fluide e comprendere contesti più ampi, grazie all'addestramento su enormi quantità di dati linguistici. I *transformer* sono un'architettura di rete neurale avanzata, utilizzata per l'elaborazione del linguaggio naturale e, pur continuando ad utilizzare il meccanismo di encoder-decoder, rappresentano un'evoluzione significativa. Infatti, invece di elaborare le

informazioni parola per parola, come avveniva nei modelli di rete precedenti, i transformer utilizzano un meccanismo chiamato *self-attention*, che consente al modello di considerare tutte le parole della sequenza contemporaneamente [25].

Spesso i chatbot moderni combinano elementi delle architetture avanzate con quelle rule-based, creando architetture ibride che sfruttano i punti di forza di entrambi gli approcci. Questi sviluppi nei chatbot e nei sistemi di dialogo rappresentano un avanzamento significativo verso l'obiettivo finale di creare sistemi di intelligenza artificiale in grado di interagire con gli esseri umani in modo naturale [13].

2.3 AIML

AIML (*Artificial Intelligence Markup Language*) è un linguaggio di programmazione basato su XML¹, sviluppato per la creazione di chatbot ruled-based. Il linguaggio fu ideato da Richard Wallace nei primi anni 2000 per supportare il progetto *A.L.I.C.E. (Artificial Linguistic Internet Computer Entity)*, ossia un chatbot nato come versione estesa di ELIZA [26]. AIML utilizza un approccio basato su pattern matching, in cui gli input dell'utente vengono confrontati con modelli di testo predefiniti chiamati *patterns*. Il sistema risponde con risposte pre-scritte, denominate *templates*, fornendo una struttura conversazionale efficiente, ma relativamente semplice rispetto alle tecnologie di IA moderne.

Un esempio tipico di AIML è rappresentato da una coppia di pattern e template, come nel seguente codice:

```
<category>
  <pattern>CIAO</pattern>
  <template>Ciao! Come posso aiutarti oggi?</template>
</category>
```

In questo esempio, quando l'utente scrive CIAO, il bot risponde con Ciao! Come posso aiutarti oggi?. La flessibilità di AIML consente di creare strutture più complesse, come risposte basate su variabili e *wildcard*². Ad esempio:

```
<category>
  <pattern>IL MIO NOME È *</pattern>
  <template>Piacere di conoscerti, <star/>!</template>
</category>
```

Qualsiasi stringa che segue IL MIO NOME È verrà catturata dal carattere jolly * e ripetuta nel template utilizzando <star/>, personalizzando così la risposta.

AIML si distingue per la sua semplicità e accessibilità. Tuttavia, questa semplicità presenta anche delle limitazioni. AIML è infatti un linguaggio deterministico, privo di capacità di apprendimento automatico proprie delle reti neurali. Nonostante ciò, rimane uno strumento adatto per applicazioni con esigenze limitate, in particolare per quelle interazioni utente-sistema che possono essere pianificate in anticipo [21].

¹L'XML (eXtensible Markup Language) è un linguaggio progettato per descrivere e organizzare dati in documenti strutturati, utilizzando un insieme di regole sintattiche che definiscono la struttura e il contenuto dei documenti tramite il markup [4].

²Un wildcard character, noto anche come metacarattere o carattere jolly, è un simbolo che, in una stringa, non rappresenta sé stesso, ma piuttosto un gruppo di altri caratteri o sequenze di caratteri [21].

2.4 Reti neurali e Large Language Model

Le reti neurali artificiali sono modelli computazionali ispirati al funzionamento del cervello umano, con l'obiettivo di simulare l'elaborazione delle informazioni da parte di neuroni biologici. Ogni rete è composta da numerosi neuroni artificiali, chiamati *perceptron*, che si combinano per formare una rete di strati. Questi strati sono connessi tra di loro attraverso collegamenti detti *pesi*, che vengono ottimizzati durante il processo di apprendimento della rete [18].

Un perceptron prende come input un insieme di valori, ciascuno moltiplicato per un peso associato. La somma pesata di questi input è definita con la variabile z e può essere espressa dalla formula:

$$z = \sum_{i=1}^n w_i x_i + b$$

dove x_i rappresenta il valore di input; w_i è il peso associato a quell'input; b è il *bias*, un termine aggiuntivo che permette al modello di essere più flessibile [12].

Il valore di z appena calcolato viene passato attraverso una funzione di attivazione, la quale rappresenta un concetto cruciale delle reti neurali, poiché introduce la non-linearità nel modello. Il ruolo della funzione di attivazione è quello di determinare l'output di ogni neurone dato z . Senza una funzione di attivazione, anche una rete neurale con più strati sarebbe un semplice modello lineare con capacità limitata. Esistono diverse funzioni di attivazione, ognuna con specifiche proprietà [12].

- **Funzione a soglia:** È la funzione di attivazione più semplice e viene utilizzata nel perceptron originale. Restituisce un output binario, attivando il neurone se la somma pesata z supera una certa soglia θ , altrimenti il neurone rimane inattivo.

$$f(z) = \begin{cases} 1 & \text{se } z \geq \theta \\ 0 & \text{se } z < \theta \end{cases}$$

- **Funzione sigmoide:** La funzione sigmoide mappa ogni valore di input z in un intervallo compreso tra 0 e 1, utile per interpretare i risultati come probabilità.

$$f(z) = \frac{1}{1 + e^{-z}}$$

- **ReLU (Rectified Linear Unit):** La funzione *ReLU* restituisce 0, se z è negativo e, z stesso, se è positivo.

$$f(z) = \max(0, z)$$

- **Funzione Tanh (Tangente Iperbolica):** La funzione *tanh* è simile alla funzione sigmoide, ma mappa i valori di input z in un intervallo tra -1 e 1.

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \tanh(z)$$

In sintesi, il compito della funzione di attivazione è trasformare la somma pesata degli input di un neurone, ossia il valore z , in un output che può essere utilizzato dai neuroni degli strati successivi.

Le reti neurali a più strati o *Multi-Layer Perceptron (MLP)* sono composte da uno strato di input, uno o più strati nascosti (*hidden layer*) e uno di output. Ogni neurone di uno strato è collegato a tutti i neuroni di quello successivo, creando una rete *fully-connected* [12].

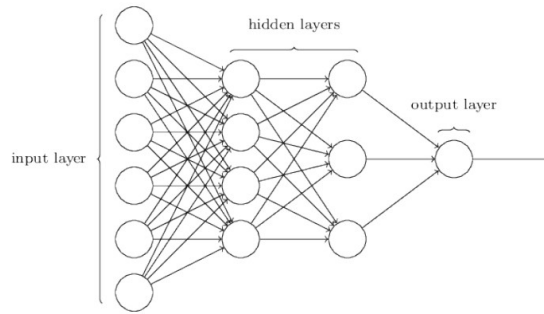


Figura 2.7. Struttura di una rete neurale con i livelli di input, nascosti e output (Fonte: [18]).

Il processo di apprendimento della rete MLP utilizza due fasi principali [12]:

- **Passata forward:** Gli input vengono propagati attraverso la rete fino allo strato di output, dove viene generato un risultato. Durante questa fase, ogni neurone combina gli input ricevuti, applica il peso e il bias, calcola z e infine applica la funzione di attivazione per determinare il suo output.
- **Passata backward:** Si confronta l'output generato con l'output desiderato e si calcola un errore. Questo errore viene poi retropropagato attraverso la rete per aggiornare i pesi con l'algoritmo della retropropagazione dell'errore. In questa fase, la funzione di attivazione gioca un ruolo chiave nel calcolo del *gradiente* che guida l'aggiornamento dei pesi.

Le reti neurali hanno rivoluzionato il campo dell'intelligenza artificiale e del machine learning, grazie alla loro capacità di apprendere pattern complessi e rappresentazioni astratte dei dati. Negli ultimi anni, una delle applicazioni più avanzate delle reti neurali è lo sviluppo dei Large Language Model, i quali utilizzano reti neurali profonde per comprendere e generare linguaggio naturale.

Large Language Model (LLM)

I Large Language Model sono modelli di apprendimento automatico, basati su reti neurali profonde, progettati per comprendere e generare linguaggio naturale su larga scala. Come detto in precedenza, essi si basano sull'architettura transformer e applicano meccanismi di attenzione per analizzare interi testi e generare risposte coerenti [25].

Come analizzato brevemente nel capitolo introduttivo, la comprensione di un LLM si basa sulla sua capacità di predire la parola successiva in una sequenza, utilizzando un approccio probabilistico. Questa previsione è fondata sulla *probabilità condizionata*, ossia sulla probabilità che una

certa parola w_t segua una sequenza di parole precedenti $w_1, w_2, \dots, w_{t-1}$. Formalmente, questa probabilità si esprime come:

$$P(w_t | w_1, w_2, \dots, w_{t-1})$$

Il modello analizza il contesto e assegna una probabilità a ciascuna parola potenzialmente successiva. Le parole con la probabilità più alta sono scelte come continuazione della sequenza.

Gli LLM utilizzano la funzione *softmax* per trasformare i punteggi grezzi in una distribuzione di probabilità, facilitando la scelta della parola più probabile:

$$P(w_t) = \frac{e^{z_t}}{\sum_i e^{z_i}}$$

dove z_t rappresenta il punteggio non normalizzato associato a ciascuna parola. Questo approccio probabilistico rende la comprensione degli LLM più simile a una previsione basata su pattern linguistici [12].

Di seguito un esempio che ha l'obiettivo di semplificare il concetto.

Consideriamo la frase *Il gatto sta dormendo su* e vogliamo predire la parola successiva. Supponiamo che il modello assegni i seguenti punteggi grezzi per tre possibili continuazioni: *letto*, *divano*, *albero*:

$$z_{\text{letto}} = 3, \quad z_{\text{divano}} = 1, \quad z_{\text{albero}} = 0$$

Per trasformare questi punteggi in probabilità, utilizziamo la funzione *softmax* definita in precedenza.

Calcoliamo ora le probabilità per ciascuna parola:

$$\begin{aligned} P(\text{letto}) &= \frac{e^3}{e^3 + e^1 + e^0} = \frac{20.09}{20.09 + 2.72 + 1} \approx 0.84 \\ P(\text{divano}) &= \frac{e^1}{e^3 + e^1 + e^0} = \frac{2.72}{20.09 + 2.72 + 1} \approx 0.11 \\ P(\text{albero}) &= \frac{e^0}{e^3 + e^1 + e^0} = \frac{1}{20.09 + 2.72 + 1} \approx 0.04 \end{aligned}$$

In questo caso, la parola *letto* ha la probabilità più alta $P(\text{letto}) = 0.84$, quindi il modello predirà che la parola successiva sarà *letto* con l'84% di probabilità.

Le allucinazioni negli LLM

La mancata comprensione dei contenuti e l'approccio probabilistico sono una delle cause principali delle cosiddette allucinazioni. Le allucinazioni negli LLM avvengono quando il modello genera informazioni che sembrano plausibili, ma che sono errate o inventate. Poiché gli LLM non hanno una conoscenza reale del mondo, ma apprendono attraverso pattern nei dati, quando si trovano a generare risposte su argomenti per i quali non hanno informazioni sufficienti, possono produrre risultati che sembrano coerenti, ma che in realtà sono falsi o non verificati. Oltre alla mancanza di comprensione, vi sono ulteriori cause che generano le allucinazioni: ad esempio i dati di addestramento del modello incompleti o inaccurati, oppure la mancanza di accesso in tempo reale a fonti di informazione [9].

Prompt Engineering

Un *prompt* è un messaggio testuale fornito a un modello di intelligenza artificiale, come un Large Language Model, per influenzare la risposta o il comportamento del modello. Rappresenta il punto di partenza per l'interazione con il modello e può essere costituito da una domanda, un'istruzione o anche, per esempio, un testo da completare. La costruzione di un prompt efficace è un processo di raffinamento che mira a migliorare la qualità delle risposte fornite dal LLM [24].

Il *Prompt Engineering* è la disciplina che si occupa di ottimizzare il testo del prompt per ottenere risposte migliori e ridurre il rischio di allucinazioni da parte di un LLM. Infatti è dimostrato empiricamente che il modo in cui si scrive il prompt influisce direttamente sulla qualità e sulla pertinenza delle risposte. Esistono diverse tecniche di prompt engineering, ognuna con vantaggi specifici [24].

- **Zero Shot:** Si fornisce al modello un'istruzione diretta senza esempi su come risolvere il compito. È una tecnica rapida, ma può risultare poco accurata per compiti complessi.
- **One/Few Shot:** Vengono forniti uno o più esempi insieme all'istruzione. Questo aiuta il modello a capire meglio il task e a rispondere in maniera più precisa.
- **Chain of Thought (CoT):** Questa tecnica richiede al modello di ragionare passo passo per risolvere un problema, simulando un ragionamento strutturato. È utile per compiti complessi che richiedono una logica sequenziale.
- **Zero Chain of Thought:** Si aggiunge una semplice frase come *Let's think step-by-step*, che stimola il modello a seguire un processo di ragionamento più riflessivo e dettagliato.
- **Contrastive Chain of Thought:** Si forniscono esempi di ragionamenti corretti e sbagliati per aiutare il modello a distinguere tra una buona e una cattiva soluzione. È utile per ridurre gli errori e migliorare la precisione [27].
- **Tree of Thoughts:** Utilizzato per problemi complessi che richiedono una visione strategica. Il modello valuta ogni passaggio del ragionamento come *sicuro*, *possibile* o *impossibile*, scartando le soluzioni errate.
- **Thread of Thoughts:** Consiste in due fasi: la prima, detta fase di ragionamento, prevede il ragionamento sui dati presentati; la seconda, detta fase di raffinamento, consiste nel generare una conclusione. Questa tecnica è utile per gestire dati complessi o caotici.
- **Least To Most:** Consiste in due fasi: la prima, detta decomposizione, ha lo scopo di dividere il task in ulteriori sotto-task; la seconda, detta risoluzione, ha l'obiettivo di risolvere i sotto-task generati.
- **Plan and Solve:** Guida il modello attraverso un piano di azione, con un insieme di passi predefiniti per risolvere un problema. Ideale per task che richiedono una pianificazione ordinata.
- **Tabular Chain of Thought:** Scompone il task principale in sotto-domande che vengono risolte una alla volta. È utile per compiti che possono essere suddivisi in passaggi logici.

Perché utilizzare ancora i chatbot rule-based?

Alla luce di quanto analizzato fino ad ora è doveroso chiedersi se i chatbot basati su regole o su corpora, siano ancora necessari o se rappresentino delle tecnologie obsolete. Ovviamente l'utilizzo di LLM ha molteplici vantaggi, principalmente legati alla generazione di nuovo linguaggio. Bisogna, tuttavia, considerare anche i seguenti fattori:

1. **Semplicità e costo:** I chatbot rule-based sono relativamente semplici da sviluppare e mantenere. Richiedono meno risorse computazionali e investimenti rispetto agli LLM, i quali necessitano di potenti infrastrutture hardware e notevoli risorse per essere addestrati e utilizzati.
2. **Controllo delle risposte:** I chatbot rule-based offrono un maggiore controllo sul comportamento del sistema. Le risposte sono prevedibili e seguono regole ben definite, riducendo il rischio di errori. Al contrario, i modelli di grandi dimensioni possono generare risposte impreviste o allucinazioni.
3. **Domini ristretti:** In contesti altamente specializzati (come ad esempio in ambito medico, scientifico o legale), i chatbot rule-based o basati su corpora possono essere più efficienti, poiché operano all'interno di un dominio limitato, minimizzando il rischio di ambiguità o errori.
4. **Privacy e sicurezza:** Gli LLM richiedono grandi quantità di dati per fornire risposte efficaci. Questo può sollevare preoccupazioni legate alla privacy, specialmente in applicazioni che trattano dati sensibili. I chatbot rule-based possono essere implementati senza l'uso di enormi quantità di dati, migliorando la protezione delle informazioni personali.
5. **Prestazioni in tempo reale:** I chatbot rule-based sono generalmente più veloci rispetto agli LLM. Non necessitano di generare linguaggio naturale in tempo reale, il che li rende più adatti in contesti in cui è richiesta una risposta immediata.

Modello LLM utilizzato

Come anticipato nel paragrafo introduttivo, nel mio sistema di traduzione viene utilizzato un Large Language Model. Nei prossimi capitoli viene spiegato in maniera più approfondita come avviene questo processo di traduzione e il ruolo chiave del modello.

In questo progetto ho utilizzato il modello Llama2 13B, sviluppato da Meta AI, che si distingue per l'uso di tredici miliardi di parametri. Questa cifra, indicata con 13B, rappresenta il numero di variabili, ossia pesi e bias, presenti all'interno del modello. Il modello è stato scaricato dalla piattaforma *Hugging Face Hub*: un importante repository di modelli di machine learning che semplifica l'accesso e la distribuzione di modelli pre-addestrati.

2.5 Linguaggio naturale controllato

Il linguaggio naturale controllato (*Controlled Natural Language*, CNL) rappresenta una versione semplificata e strutturata del linguaggio naturale, progettata per ridurre o eliminare le ambiguità e le complessità tipiche del linguaggio umano. Questo tipo di linguaggio è spesso utilizzato in contesti dove la precisione e la coerenza sono fondamentali. Il CNL mira a facilitare la comprensione da parte delle macchine, pur rimanendo accessibile agli esseri umani, unendo insieme la semplicità, propria del linguaggio formale, con la naturalezza, propria del linguaggio umano.

Capitolo 3

Descrizione e implementazione dell'architettura di traduzione

3.1 Descrizione generale del sistema

Il sistema in questione si basa su un processo iterativo e multi-fase per tradurre l'input iniziale dell'utente in un formato coerente per il chatbot AIML NoVAGraphS. Questa sezione, oltre a fornire una panoramica dell'architettura illustrando i principali componenti, descrive anche l'implementazione del sistema sviluppato per migliorare la pertinenza delle risposte del chatbot NoVAGraphS.

Il progetto è stato implementato utilizzando *Google Colab*, il quale è uno strumento basato su cloud che consente di eseguire codice Python all'interno di un ambiente *Jupyter Notebook*. Offre risorse di calcolo potenti, come la *GPU*¹, senza la necessità di installare software localmente. Grazie alla sua natura collaborativa, più utenti possono lavorare sullo stesso notebook contemporaneamente, con i file che possono essere salvati direttamente su *Google Drive*. Colab è particolarmente utile per progetti che richiedono un'elevata capacità computazionale, come il machine learning e il deep learning [22].

Il codice sorgente, i file di dati e le analisi relative al progetto sono disponibili nel repository *GitHub* all'indirizzo https://github.com/davidecandel8/NoVAGraphS_Translation. L'intero processo di traduzione può essere visionato direttamente su Google Colab accedendo al notebook disponibile al seguente indirizzo: <https://colab.research.google.com/drive/1x6KyPfPgGVdUJzgdGtH0mC3pnEldiLN-?usp=sharing>.

¹La GPU (*Graphics Processing Unit*) è un'unità di elaborazione grafica, progettata per eseguire calcoli in parallelo su grandi quantità di dati, rendendola ideale per applicazioni di deep learning [20].

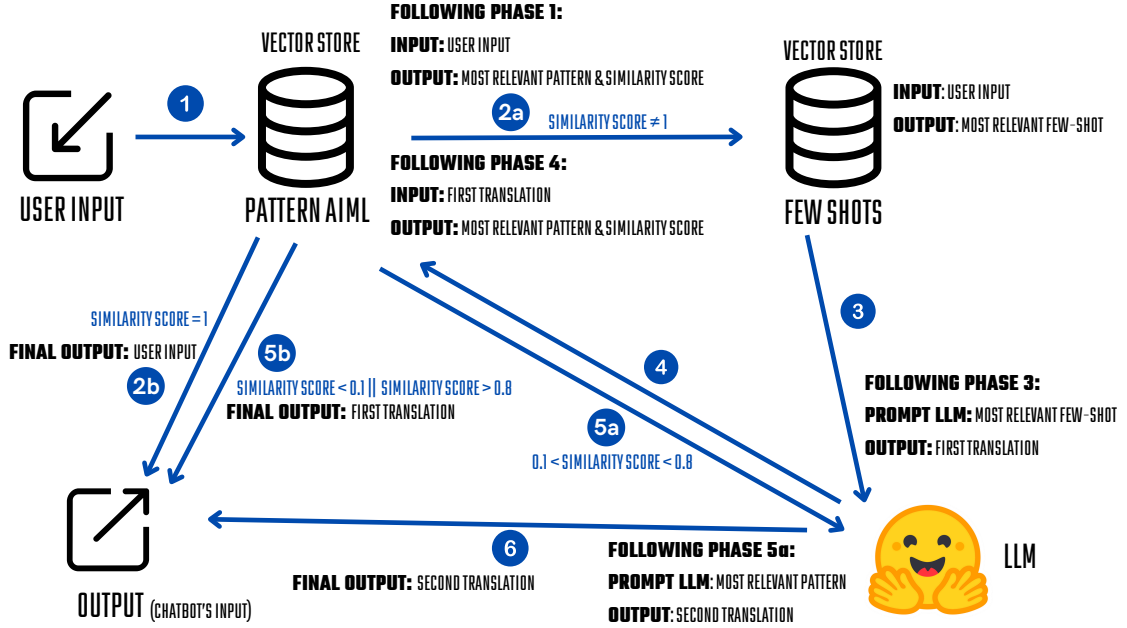


Figura 3.1. Rappresentazione complessiva dei possibili percorsi dell'input nell'architettura di traduzione. L'immagine mostra le diverse fasi di verifica della pertinenza e di traduzione, evidenziando come il sistema gestisca l'input in base al punteggio di similarità e alla necessità di traduzioni successive.

Come mostrato dalla Figura 3.1, gli strumenti chiave per l'implementazione del sistema di traduzione sono i seguenti:

- **Input dell'utente:** L'input in linguaggio naturale che l'utente inserisce attraverso l'interfaccia del chatbot. Questo costituisce il punto di partenza per l'intero processo di traduzione e risposta. Maggiori dettagli sull'input dell'utente e sul suo trattamento sono descritti nella Sezione 3.2.
- **Vector Store con pattern AIML:** Un *vector store*, o vector database, è un database progettato per memorizzare e gestire vettori, ossia rappresentazioni numeriche di dati. In questo contesto, memorizza le rappresentazioni numeriche dei pattern AIML utilizzati per costruire il chatbot NoVAGraphS. Diversamente dai database relazionali, che richiedono la formulazione di una query in linguaggio SQL per recuperare i dati desiderati, nei vector store è sufficiente fornire un input per ottenere i risultati. Il sistema converte l'input in un vettore numerico, attraverso tecniche di *embedding semantico*. Il vector store restituisce quindi il pattern AIML più pertinente rispetto all'input fornito, insieme a un *similarity score* che misura il grado di somiglianza tra il vettore dell'input e i vettori memorizzati [10]. Nella Sezione 3.3 viene approfondito nel dettaglio l'utilizzo dei vector store.

Inoltre, come in parte anticipato nella Sezione 2.3, i pattern AIML possono essere considerati una forma semplificata di espressioni regolari. Le espressioni regolari (*regex*) sono sequenze di simboli che descrivono modelli specifici all'interno di stringhe di testo, consentendo operazioni di ricerca, sostituzione o validazione di elementi come numeri, parole o formati complessi [7]. Infatti, grazie all'uso del carattere wildcard (*), i pattern AIML possono adattarsi a diverse varianti di input, permettendo al sistema di riconoscere schemi di frasi che condividono somiglianze strutturali, ma presentano variazioni nei dettagli specifici.

- **Vector Store con esempi few-shot:** Un vector store che contiene un insieme di esempi predefiniti (few-shot). Questo vector store, sulla base dell'input fornito, restituisce gli esempi più pertinenti che fungono da guida per il modello durante la traduzione dell'input, migliorando la qualità delle risposte generate dal Large Language Model. La struttura e il ruolo degli esempi few-shot sono spiegati in dettaglio nella Sezione 3.4.
- **Large Language Model:** Un modello di linguaggio di grandi dimensioni utilizzato per tradurre l'input dell'utente in linguaggio naturale controllato, con l'obiettivo di semplificare l'interazione e migliorare la pertinenza delle risposte generate dal chatbot. Il funzionamento del Large Language Model e il suo ruolo nell'architettura sono trattati in dettaglio nella Sezione 3.5.
- **Output finale:** L'output finale rappresenta la risposta fornita dal sistema al termine del processo di traduzione. L'output inoltre è l'input stesso fornito al chatbot NoVAGraphS.

Nella Figura 3.1 sono rappresentati tutti i possibili percorsi che l'input dell'utente può seguire all'interno dell'architettura di traduzione, fornendo una panoramica completa del sistema. L'immagine mostra come il flusso venga gestito in base alla pertinenza dell'input e alla necessità di eventuali traduzioni. Le successive figure 3.2, 3.3 e 3.4 presentano, invece, i singoli percorsi separatamente, concentrandosi su scenari specifici e tralasciando le alternative presenti nella figura principale.

Il sistema di traduzione inizia con l'inserimento della domanda da parte dell'utente in lingua inglese nell'apposito spazio dell'interfaccia grafica. Viene verificato che l'input inserito dall'utente possa essere compreso dal chatbot senza la necessità di una traduzione in linguaggio naturale controllato. A tale scopo, viene utilizzato il primo vector store dentro il quale sono presenti tutti i pattern AIML, utilizzati per la costruzione del chatbot. Il vector store restituisce il pattern AIML più rilevante e il corrispondente similarity score, ossia la misura numerica della somiglianza tra l'input e il pattern. Per ulteriori informazioni, fare riferimento alla Sezione 3.3. Come è illustrato in Figura 3.2, se il similarity score è pari ad uno, l'input coincide esattamente con un pattern AIML, ossia intercetta la sua espressione regolare. Di conseguenza la richiesta dell'utente si può considerare ben formata e il chatbot riesce a rispondere in modo pertinente.

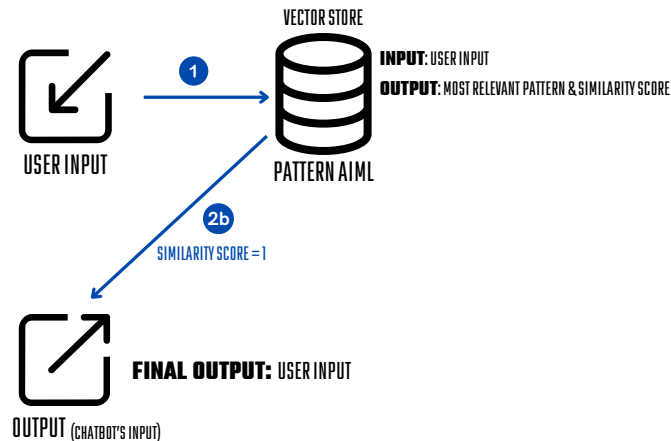


Figura 3.2. Architettura del sistema quando l'input dell'utente è correttamente formato. In questo scenario, l'input corrisponde esattamente a un pattern AIML, ottenendo un punteggio di similarità pari a uno, permettendo al chatbot di rispondere direttamente senza ulteriori traduzioni.

Altrimenti, come mostrato in Figura 3.3, se il similarity score è diverso da uno e dunque l'input non corrisponde direttamente ad un pattern AIML, allora il sistema traduce l'input da linguaggio naturale a linguaggio naturale controllato. Per svolgere questo processo di traduzione viene utilizzato il secondo vector store, diverso dal precedente. Come anticipato, all'interno del vector store è contenuto un insieme di esempi, detti anche few-shot, da includere nel corpo del prompt. Il prompt guida il Large Language Model nella generazione di una traduzione coerente con gli esempi restituiti dal vector store. Per ulteriori informazioni vedere la Sezione 3.4 e la Sezione 3.5.

La frase tradotta, sulla base degli esempi forniti, viene quindi re-inviata al primo vector store per determinare il pattern AIML più pertinente e il corrispondente similarity score. Se il similarity score è molto vicino a zero, indicando scarsa pertinenza, o molto vicino a uno, indicando un'alta pertinenza, la frase tradotta viene comunque utilizzata come output finale del sistema di traduzione e inviata come input al chatbot. Questo processo è illustrato in Figura 3.3. La scelta di accettare anche input con bassa pertinenza si basa sul fatto che una query dell'utente può essere ben formulata oppure mal strutturata. In quest'ultimo caso, è corretto che il chatbot risponda con un messaggio di errore o informi l'utente della mancata comprensione.

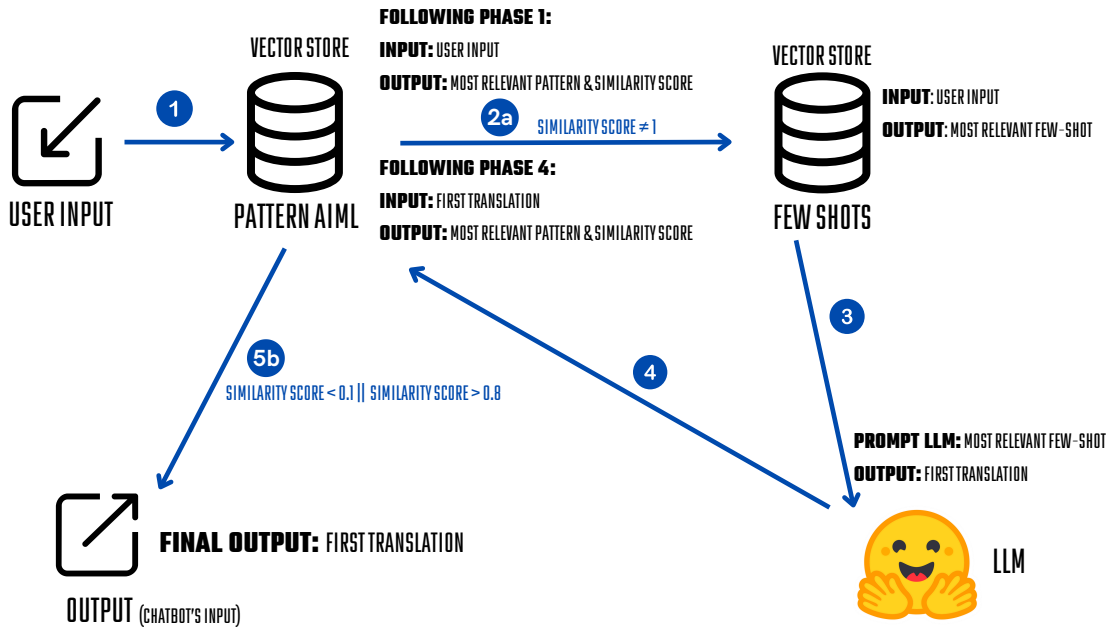


Figura 3.3. Architettura del sistema in relazione alla qualità della traduzione prodotta dal Large Language Model, che può risultare sia accurata che totalmente inesatta. In questo percorso, se la prima traduzione mostra una coerenza molto alta o molto bassa con i pattern AIML, viene accettata come output finale, permettendo al sistema di rispondere anche in caso di input mal formulati.

Come illustrato dalla Figura 3.4, se la prima traduzione dell'utente presenta una buona, ma non perfetta, corrispondenza con il pattern, allora il sistema effettua una seconda traduzione utilizzando nuovamente il Large Language Model. In questo caso, il prompt contiene il pattern AIML più pertinente restituito dal vector store, che rappresenta l'espressione regolare da seguire, affinché il chatbot possa rispondere correttamente. Questa seconda traduzione rappresenta l'output del sistema di traduzione e viene utilizzata come input per il chatbot rule-based NoVAGraphS. Per approfondimenti vedere la Sezione 3.5.

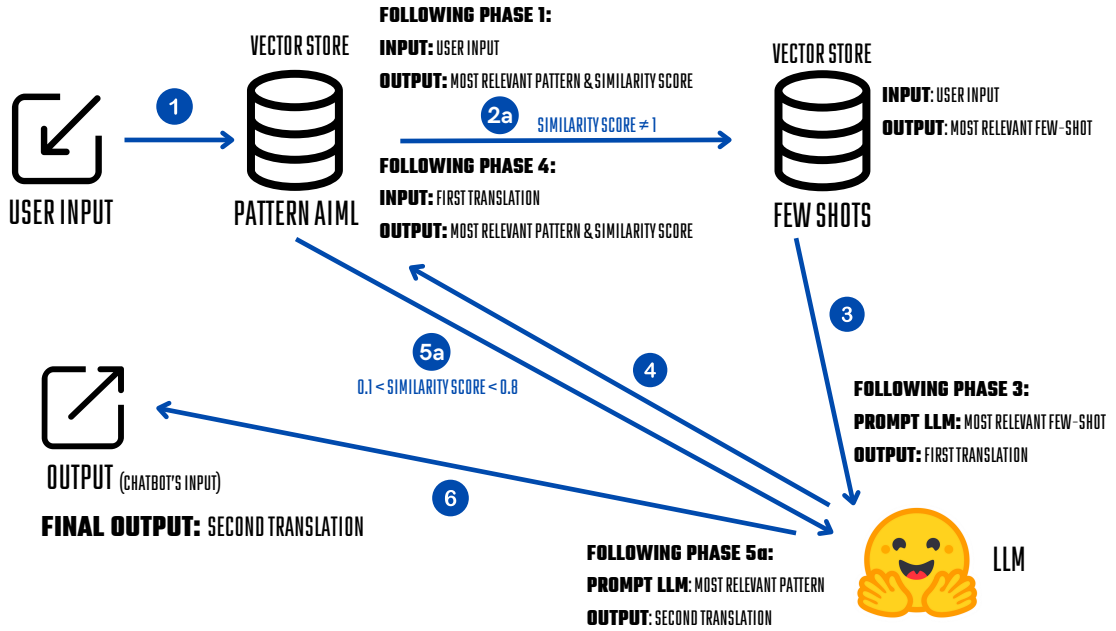


Figura 3.4. Architettura del sistema con una seconda traduzione fornita dal Large Language Model. Se la prima traduzione presenta una corrispondenza buona ma non perfetta con il pattern AIML, il sistema esegue una seconda traduzione, utilizzando il pattern più pertinente come guida per il modello.

3.2 Gestione dell'input dell'utente con il server Flask mediante ngrok

L'input dell'utente consiste in un testo che rappresenta la richiesta rivolta al chatbot NoVAGraphS, specializzato nel campo degli automi a stati finiti. Questo testo può variare in complessità e contenuto, con l'unico requisito che sia in formato testuale e redatto in inglese. L'idea di questa architettura è proprio quella di rendere il più flessibile possibile l'input dell'utente. Questo evita all'utente lo sforzo di "indovinare" o prevedere i possibili pattern che il programmatore del chatbot ha pensato, lasciando all'utente la più totale libertà nel porre la propria domanda. Si tratta di un modo per superare i limiti progettuali che un normale chatbot rule-based può avere.

L'interfaccia grafica sviluppata, grazie alla quale l'utente può interagire con il chatbot, è collegata a Google Colab utilizzando *ngrok*, uno strumento che consente di creare un tunnel sicuro tra un server locale e Internet. In questo progetto, il server Flask viene eseguito all'interno di un notebook su Google Colab, che non è accessibile direttamente dall'esterno. Ngrok è stato utilizzato per risolvere questo problema: crea un *URL* pubblico temporaneo che reindirizza il traffico Internet verso il server locale eseguito su Colab. Questo metodo si è reso necessario perché, senza ngrok, non sarebbe possibile la comunicazione tra l'interfaccia e il notebook di Colab. La connessione stabilita tramite ngrok garantisce inoltre che i dati siano trasmessi attraverso un canale *HTTPS* sicuro [17].

3.3 Verifica della pertinenza mediante vector store

Talvolta l'input dell'utente è già ben formato, dunque non è necessario che vi sia un processo di traduzione per modificare la richiesta. Pertanto, per verificare la correttezza di una domanda, viene utilizzato un vector store. Come spiegato in precedenza, all'interno del vector store sono presenti tutti i pattern AIML, utilizzati per costruire il chatbot. A differenza dei database relazionali, dove è necessario scrivere una query in linguaggio SQL per ottenere i valori desiderati, nei vector store è sufficiente fornire un input. Il sistema converte l'input in un vettore numerico, attraverso tecniche di *embedding semantico* e restituisce automaticamente le tuple più simili sulla base di metriche di similarità, come la *cosine similarity* o la *distanza euclidea*. Un embedding semantico è una rappresentazione numerica di parole, frasi o documenti che cattura il loro significato. Oggetti semanticamente simili sono trasformati in vettori vicini tra loro nello spazio vettoriale, mentre quelli diversi sono più distanti. Queste rappresentazioni permettono al sistema di confrontare l'input dell'utente con i dati presenti nel vector store in modo efficiente [10, 23].

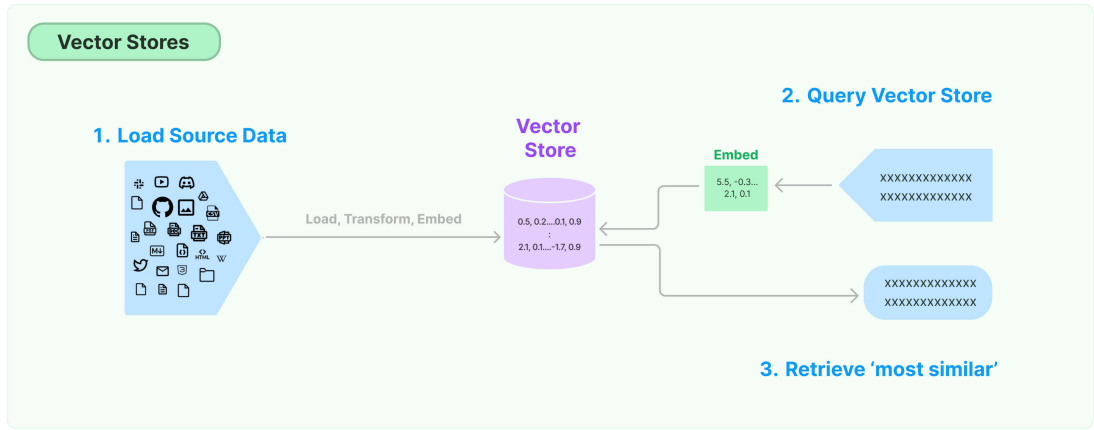


Figura 3.5. Schema del funzionamento di un vector store (Fonte: [14]).

Le metriche più comuni per calcolare la similarità tra vettori sono [16]:

- **Similarità Coseno:** La similarità coseno è una delle metriche più comuni per calcolare la similarità tra due vettori. Questa metrica ha lo scopo di calcolare in particolare l'angolo tra due vettori nello spazio vettoriale. La formula è:

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \times \|\mathbf{B}\|}$$

dove $\mathbf{A}$ e $\mathbf{B}$ sono i vettori dei due testi; $\mathbf{A} \cdot \mathbf{B}$ è il prodotto scalare dei due vettori; $\|\mathbf{A}\|$ e $\|\mathbf{B}\|$ sono le lunghezze dei vettori. Il risultato della cosine similarity è un valore compreso tra -1 e 1, con 1 che indica una perfetta similarità, 0 assenza di similarità, e -1 opposizione totale tra i vettori.

- **Distanza Euclidea:** A differenza della cosine similarity, che misura l'angolo tra due vettori, la distanza euclidea misura la distanza effettiva tra i punti corrispondenti ai vettori nello

spazio vettoriale. La formula per la distanza euclidea tra due vettori **A** e **B** è:

$$\text{Distanza Euclidea} = \sqrt{\sum_{i=1}^n (\mathbf{A}_i - \mathbf{B}_i)^2}$$

dove n è il numero di dimensioni del vettore; $\mathbf{A}_i$ e $\mathbf{B}_i$ rappresentano i valori delle singole componenti dei vettori. Maggiore è la distanza, meno simili sono i due vettori. Per ottenere un punteggio di similarità a partire dalla distanza euclidea, spesso si utilizza l'inverso della distanza normalizzata, come segue:

$$\text{Similarity Score} = \frac{1}{1 + \text{Distanza Euclidea}}$$

Se l'input dell'utente rispettasse l'espressione regolare identificata dal pattern, allora il valore attuale di similarity score viene sostituito con il valore 1. Il sistema riconosce il valore 1 come corrispondenza perfetta, dunque l'input dell'utente diventa l'output finale dell'architettura di traduzione, nonché l'input per il chatbot.

Implementazione della verifica della pertinenza

L'implementazione della verifica della pertinenza si basa su una combinazione di tecniche di embedding semantico e corrispondenza tramite wildcard, come descritto nella sezione precedente. Di seguito vengono descritti i principali passaggi implementativi.

Il primo passo consiste nell'estrazione dei pattern dal file AIML, utilizzato per la costruzione del chatbot NoVAGraphS. I pattern vengono individuati all'interno dei tag `<pattern>` e `</pattern>` utilizzando un'espressione regolare. Una volta estratti, i pattern vengono pre-elaborati per rimuovere i caratteri jolly (*), così da prepararli per l'operazione di embedding. Rimuovere i caratteri jolly consente di semplificare il confronto semantico tra l'input e i pattern, eliminando le complessità derivanti dalla gestione di wildcard e permettendo di confrontare direttamente i vettori dei pattern preprocessati.

Successivamente, i pattern pre-elaborati (senza *) vengono convertiti in vettori numerici utilizzando il modello di embedding **SentenceTransformer** con la variante **paraphrase-MiniLM-L6-v2**. Questo modello genera rappresentazioni semantiche dei pattern sotto forma di vettori. I vettori vengono inseriti in un vector store basato su **FAISS**², una libreria che permette di cercare i vettori più simili in base a una query fornita dall'utente [6].

Quando l'utente fornisce un input, il sistema lo converte in un vettore numerico utilizzando lo stesso modello di embedding. Questo vettore viene confrontato con quelli dei pattern AIML presenti nel vector store, restituendo i pattern più vicini in termini di distanza euclidea. Tuttavia, poiché la ricerca tramite embedding si basa su similarità semantica, i risultati ottenuti necessitano di un ulteriore affinamento. A questo punto, per confrontare l'input con i pattern originali, vengono reintrodotti i caratteri jolly, non più come wildcard generiche, ma come espressioni regolari. Pertanto si convertono i caratteri jolly dei pattern originali in espressioni regolari, consentendo di verificare se l'input dell'utente corrisponde esattamente a uno dei pattern originali. Se viene trovata una corrispondenza esatta, al pattern viene attribuito un punteggio di similarità pari a 1, indicando una perfetta corrispondenza. Se invece non si trova una corrispondenza esatta, il sistema restituisce il pattern più simile, in base alla distanza calcolata nel vector store.

²FAISS (Facebook AI Similarity Search) è una libreria open-source sviluppata da Facebook AI Research (FAIR) per la ricerca efficiente di vettori simili in grandi insiemi di dati [6].

3.4 Ricerca di esempi pertinenti (few-shot) mediante vector store

Nel contesto di questa architettura, la tecnica di few-shot prompting è impiegata per migliorare l'accuratezza e la pertinenza delle risposte generate dal sistema. Few-shot prompting è una tecnica di prompting engineering che consiste nell'includere pochi esempi rilevanti all'interno del prompt per mostrare al modello come dovrebbe rispondere a un input simile. Con questa tecnica si vuole dunque che il modello generalizzi correttamente i nuovi input, basandosi sugli esempi forniti [24]. Gli esempi few-shot sono stati modellati ad hoc affinché il LLM riesca a compiere la traduzione dell'input dell'utente, in linguaggio naturale controllato, nel modo più pertinente possibile. Per modellare gli esempi, si è analizzato un corpus di interazioni fra utenti e il chatbot NoVAGraphS, privo del processo di traduzione dell'input. Questo corpus, disponibile nella cartella `first_evaluation` della repository GitHub menzionata nella Sezione 3.1, contiene una serie di conversazioni in cui il chatbot risponde a diverse domande poste dagli utenti. Il corpus, infatti, include gli input dell'utente, le risposte del chatbot ed una classificazione degli errori commessi. Gli errori sono stati suddivisi in diverse categorie, fornendo una base per comprendere i fallimenti sistematici del chatbot.

Il primo ostacolo è stato quello di scegliere quale tecnica di prompting fosse la più pertinente da utilizzare in questo contesto. Sulla base degli errori evidenziati dal corpus, sono state applicate diverse tecniche di prompting engineering, come documentato nel notebook disponibile al seguente indirizzo: https://colab.research.google.com/drive/18r9xysGLpahR_TgCnOVXDIYXg7im1Mfe. In particolare, sono state utilizzate le seguenti tecniche con un Large Language Model per poter tradurre, in linguaggio naturale controllato, gli input degli utenti che generano un errore nel corpus:

- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Chain of thoughts prompting
- Zero chain of thoughts prompting

Ogni tecnica è stata applicata per correggere specifici tipi di errore e le correzioni sono state valutate in base alla pertinenza delle risposte generate. I risultati di queste tecniche, disponibili nella cartella `prompt_engineering` della repository GitHub, mostrano chiaramente che la tecnica di few-shot prompting è risultata la più efficace nel migliorare la qualità delle risposte del chatbot.

Dopo aver stabilito che la tecnica few-shot è la più adatta nel nostro contesto, si è proceduto alla modellazione degli esempi. Questi esempi sono stati creati e indicizzati con lo scopo di proporre un input alternativo a tutte quelle richieste a cui il chatbot NoVAGraphS generava una risposta errata. All'interno del sistema, infatti, è presente un vector store che contiene i vettori rappresentativi di una vasta gamma di esempi predefiniti.

Analogamente a quanto analizzato prima, quando l'utente inserisce una query, questa viene trasformata in un vettore numerico utilizzando tecniche di embedding. Questo vettore cattura la semantica della query, rendendola confrontabile con i vettori degli esempi nel database. Utilizzando una metrica di similarità, il sistema calcola quanto ciascun esempio sia simile al vettore della query. Gli esempi con il punteggio di similarità più alto vengono considerati come i più pertinenti.

Sulla base di questo punteggio di similarità, il sistema seleziona un insieme ristretto di esempi che sono utilizzati per guidare la risposta del modello.

Di seguito un esempio pratico per esemplificare il concetto. Supponiamo che l'utente inserisca il seguente input:

Give me the number of nodes

Di fronte a questo input, il chatbot risponderebbe con una richiesta di chiarimento o un messaggio di errore. Questo avviene perché il chatbot, essendo basato su regole, non è in grado di fornire una risposta pertinente a questa domanda, in quanto non riconosce la parola *nodes* come pattern AIML predefinito. Pertanto è necessario, attraverso un Large Language Model, modificare e tradurre la query dell'utente. Il sistema confronterà quindi questa richiesta con gli esempi predefiniti, identificando quelli più pertinenti sulla base della similarità semantica. Di seguito sono riportati gli esempi trovati più pertinenti all'input iniziale:

- Input: *What's the number of nodes*
Output: *How many states*
- Input: *How many nodes are there*
Output: *How many states are there*
- Input: *Nodes*
Output: *Tell me more about states*
- Input: *Give me a list of the arcs*
Output: *What are the arcs*
- Input: *Tell me a little bit about the automaton*
Output: *Describe me briefly the automaton*
- Input: *Maximum number of states in an automaton*
Output: *How many states are there*
- Input: *And how about the arrows*
Output: *What are the arcs*
- Input: *If the input is 110 which is the result*
Output: *Describe me the automaton*

In questo esempio, gli input e output elencati servono come esempi few-shot che il modello utilizza per guidare la generazione della risposta. Gli esempi sono modellati dividendo la frase in input, ossia la frase dell'utente che genera un errore, e output, ossia la frase tradotta in linguaggio naturale controllato sulla base delle regole AIML. Il sistema mostra al modello come tradurre input simili, come *Give me the number of nodes*, in una risposta pertinente come *How many states*. Questo approccio permette al modello di apprendere come affrontare nuove query, basandosi su esempi precedentemente indicizzati, migliorando così la qualità e la pertinenza delle risposte.

Implementazione della ricerca di esempi pertinenti

L'implementazione di questo approccio si basa sull'uso di un vector store in combinazione con tecniche di embedding semantico per facilitare la ricerca degli esempi più pertinenti rispetto all'input dell'utente. Per realizzare ciò, è stato utilizzato lo stesso modello di embedding usato per la verifica della pertinenza. Il modello `SentenceTransformer('paraphrase-MiniLM-L6-v2')` trasforma gli esempi predefiniti in vettori numerici, consentendo di misurare la similarità semantica tra frasi. Anche in questo contesto si utilizza la libreria FAISS, che permette di eseguire ricerche rapide di somiglianza tra vettori [6]. In questo modo, ogni input presente nel corpus degli esempi è rappresentato da un vettore nello spazio numerico, rendendolo confrontabile con altri vettori. Quando l'utente inserisce una query, questa viene anch'essa trasformata in un vettore numerico, utilizzando il medesimo modello di embedding. Il sistema effettua quindi una ricerca nell'indice FAISS per trovare gli input degli esempi più simili alla query. Questo processo avviene con il seguente codice:

```
def search_io_vector_store(translation):
    query_vector = embedding_model.encode([translation])[0].astype(np.float32)
    # Numero di risultati da restituire
    k = 8

    # Esecuzione della ricerca
    distances, indices = index_io.search(query_vector.reshape(1, -1), k)

    # Recupero dei risultati
    results = []
    for idx in indices[0]:
        result = metadata_io[idx]
        results.append(result)
        if len(results) >= k:
            break
    return results
```

Questa funzione cerca nell'indice i vettori degli input più vicini a quello della query inserita dall'utente, restituendo anche i corrispondenti output predefiniti. Le coppie input-output ottenute rappresentano gli esempi few-shot utilizzati per guidare la risposta del modello.

L'utilizzo di questa tecnica di ricerca basata sulla similarità semantica garantisce che gli input dell'utente, anche se non perfettamente corrispondenti a un pattern AIML predefinito, possano essere tradotti in query simili con esempi pertinenti. Ciò migliora l'accuratezza delle risposte del chatbot, specialmente in quei casi dove la richiesta dell'utente non coincide esattamente con i pattern presenti nel sistema.

3.5 Traduzione in CNL mediante LLM

Il processo di traduzione da linguaggio naturale a linguaggio naturale controllato avviene per mezzo di un Large Language Model. Come presentato in precedenza, il modello utilizzato è `LLama2-13B`, il quale è stato configurato per essere eseguito su GPU, come mostrato nel seguente frammento di codice:

```
# Seleziona LLama2 13B come modello
```

```
model_name_or_path = "TheBloke/Llama-2-13B-chat-GGML"
model_basename = "llama-2-13b-chat.ggmlv3.q5_1.bin"

# Scarica il modello dal Hugging Face Hub
model_path = hf_hub_download(repo_id=model_name_or_path, filename=model_basename)

# Inizializza il modello con configurazione per GPU
lcpp_llm = Llama(
    model_path=model_path,
    n_threads=2,
    n_batch=512,
    n_gpu_layers=32
)
```

Il processo di traduzione si sviluppa in tre momenti differenti.

1. **Prima traduzione da linguaggio naturale a CNL:** I few-shot più pertinenti, trovati interrogando il vector store, vengono utilizzati come esempi che il Large Language Model deve seguire per tradurre correttamente l'input dell'utente. Il sistema pertanto genera una prima traduzione dell'input in CNL, semplificando il linguaggio e riducendo le ambiguità, sulla base proprio del prompt scritto e degli esempi forniti. Il template utilizzato per costruire il prompt, `llama2PromptTemplate`, ha la seguente struttura:

```
llama2PromptTemplate = lambda systemPrompt, userPrompt: f"""
<s>[INST] <<SYS>>
{systemPrompt}
<</SYS>>

{userPrompt} [/INST]
"""
```

Questo template aiuta a fornire al modello istruzioni chiare e un contesto ben delimitato. Le componenti `<<SYS>>` e `<</SYS>>` delimitano il prompt di sistema, che specifica al modello il compito da eseguire. Invece il prompt dell'utente contiene l'input specifico da tradurre. Questa struttura favorisce risposte più coerenti e accurate, mantenendo il focus sul contesto fornito [5].

Nel caso della traduzione iniziale, il `systemPrompt` e lo `userPrompt` sono definiti come segue:

```
systemPromptFewShots = """
You are an expert on finite state automata. Translate user inputs to a
controlled natural language. Use precise, technical terminology, and
format the response as a JSON object with the translation inside the
'Translation' field.
"""

userPromptFewShots = lambda text, example_list: f"""
Please translate the following inputs into controlled natural language.
Use the format provided in the examples:
{format_examples(example_list)}
Now, translate the new input using the same controlled natural
```

```
language format. Format the response as a JSON object with the
translation inside the 'Translation' field.
```

```
Input: "{text}"
```

```
Output:
```

```
"""
```

In questo modo, il modello utilizza esempi specifici per tradurre l'input dell'utente in linguaggio naturale controllato. Questi esempi sono forniti nel formato JSON, che facilita la comprensione e la manipolazione delle traduzioni da parte del sistema.

2. **Verifica della prima traduzione:** La traduzione in CNL potrebbe non essere completamente comprensibile per il chatbot. Pertanto, è necessario verificare l'accuratezza della traduzione. Questo avviene utilizzando la stessa tecnica descritta nella Sezione 3.3. In questo contesto l'input fornito al vector store, contenente i pattern AIML, è proprio la traduzione generata dal LLM.

Come espresso precedentemente, al termine di questa fase, si ottiene il pattern AIML più pertinente rispetto all'input e il relativo similarity score. Se la traduzione è ben formata, con un valore di similarity score vicino all'uno, non è necessario un ulteriore perfezionamento. Al contrario, se la traduzione non è ancora del tutto comprensibile per il chatbot, si procede con una seconda traduzione per migliorarla. Infine, se la traduzione generata dal LLM risultasse del tutto incomprensibile per il chatbot, ossia con un valore di similarity score molto vicino allo zero, la traduzione viene lasciata invariata. Questo evita che il sistema tenti di forzare la traduzione di un input privo di senso in qualcosa di comprensibile per il chatbot. Infatti, se l'input non ha significato nel contesto specifico, è opportuno che il chatbot risponda con un messaggio di errore o richieda chiarimenti. In questo caso, la prima traduzione diventa l'output finale del sistema e viene utilizzata come input per il chatbot NoVAGraph.

3. **Perfezionamento della prima traduzione con una seconda traduzione combinata con AIML:** Come analizzato in precedenza, talvolta la prima traduzione non è del tutto comprensibile da parte del chatbot. In questo senso è necessario che il Large Language Model si occupi di generare una traduzione più focalizzata alle regole sintattiche che il chatbot sia in grado di analizzare. Per fare ciò si utilizza proprio il pattern AIML più pertinente rispetto alla prima traduzione, ritornato dal vector store durante la fase di verifica. L'obiettivo del modello di linguaggio è quindi quello di modificare la prima traduzione, mantenendo invariati i contenuti, ma adattando la sintassi della frase per renderla compatibile con le regole AIML. Si desidera infatti ottenere una query che richieda le stesse informazioni della prima traduzione, ma che rispetti la struttura sintattica del pattern più pertinente. Il LLM genera pertanto una traduzione che rappresenta l'output finale del sistema e l'input tradotto per il chatbot.

Anche in questa fase viene utilizzato un prompt strutturato per guidare il modello nella traduzione. Di seguito sono riportati il `systemPrompt` e lo `userPrompt` per la traduzione mediante pattern AIML:

```
systemPrompt_AIML_LNC = """
```

```
You are an assistant that transforms questions into a controlled natural
language format using a regular expression. Respond only with the JSON
object containing the translation. Use the regular expression to ensure
the structure is correct. Do not include any additional text.
```

```
"""
```

```
userPrompt_AIML_LNC = lambda translation, regular_expression: f"""
Translate the input into a controlled natural language using the regular
expression. Respond only with the JSON object.
```

Example 1:

```
Input: "what is the initial stage"
Regular Expression: INITIAL STATE
JSON Translation: {{
  "Translation": "what is the initial state"
}}
```

Example 2:

```
Input: "tell me the transitions"
Regular Expression: TRANSITIONS
JSON Translation: {{
  "Translation": "transitions"
}}
```

Example 3:

```
Input: "What the final state is?"
Regular Expression: * FINAL STATE
JSON Translation: {{
  "Translation": "what is the final state"
}}
```

Task:

```
Input: {translation}
Regular Expression: {regular_expression}
Provide the JSON translation below:
"""
```

In questo caso, l'uso di espressioni regolari garantisce che la traduzione rispetti le regole sintattiche necessarie per essere compresa dal chatbot. Il modello genera una nuova traduzione che, pur mantenendo invariati i contenuti, adatta la struttura della frase per renderla compatibile con i pattern AIML.

3.6 Output finale

Il processo di traduzione si conclude con la generazione dell'output finale, che rappresenta la risposta elaborata dal sistema alla query dell'utente. Questo output è il risultato di tutte le fasi precedenti, integrate in un'unica risposta, il più pertinente possibile. La frase finale tradotta coincide con la nuova query a cui il chatbot NoVAGraph deve rispondere.

Capitolo 4

Valutazione del sistema e analisi dei risultati

4.1 Valutazione delle performance del sistema

In questo capitolo vengono esaminate le modalità di valutazione del sistema implementato. Le seguenti figure mostrano l'interfaccia utente prima e dopo la fase di valutazione. La valutazione del sistema è stata eseguita attraverso due approcci distinti: il feedback degli utenti e la valutazione esperta.

Question:

Send

Response 1: I dont' know. Try asking something like, "Which is the initial state?"

Response 2: There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both initial and final state.

History of questions and answers

#	Question	Answer	Rating	Expert Rating
1	give me the number of nodes	Response 1: I dont' know. Try asking something like, "Which is the initial state?"	1 2 3 4 5	1 2 3 4 5
		Response 2: There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both initial and final state.	1 2 3 4 5	1 2 3 4 5

Figura 4.1. Interfaccia utente prima dell'avvio del processo di valutazione. La *Response 1* rappresenta la risposta diretta all'input dell'utente non tradotto, mentre la *Response 2* è generata a seguito della traduzione dell'input in *How many states are there?*.

Question:

Send

Response 1: I dont' know. Try asking something like, "Which is the initial state?"

Response 2: There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both initial and final state.

History of questions and answers

#	Question	Answer	Rating	Expert Rating
1	give me the number of nodes	Response 1: I dont' know. Try asking something like, "Which is the initial state?"	Rating: 2	Expert Rating: 1
		Response 2: There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both initial and final state.	Rating: 4	Expert Rating: 5

Figura 4.2. Interfaccia utente successiva al completamento del processo di valutazione. La *Response 1* rappresenta la risposta all'input originale dell'utente, mentre la *Response 2* risponde all'input tradotto in *How many states are there?*.

Valutazione dell'utente

La valutazione delle performance del sistema è stata condotta attraverso il feedback dell'utente sulla base della pertinenza delle risposte che il chatbot genera alle richieste. Gli utenti, tramite l'apposita interfaccia, possono valutare la risposta fornita dal chatbot su una scala da 1 a 5. L'interfaccia in questione non solo mostra la risposta del chatbot rispetto alla richiesta tradotta, ma mostra anche la risposta del chatbot all'iniziale input dell'utente, senza la traduzione. La visualizzazione sia della risposta del chatbot prima della traduzione, sia di quella successiva alla traduzione, è stata progettata per consentire un confronto efficace e verificare se il sistema di traduzione migliori effettivamente la pertinenza delle risposte.

La scala di valutazione si articola come segue:

1. **Risposta completamente irrilevante:** Il chatbot non ha compreso la mia richiesta e la risposta fornita è completamente irrilevante rispetto alle mie esigenze. Per poter ricevere la risposta ho bisogno di riformulare la domanda al chatbot.
2. **Risposta poco pertinente:** Il chatbot ha compreso solo in parte la mia domanda. La risposta fornita accenna vagamente all'argomento principale della mia richiesta. Potrei aver bisogno di una domanda integrativa per ottenere quello che ho richiesto.
3. **Risposta buona:** Il chatbot ha compreso abbastanza la mia domanda. La risposta fornita non è esaustiva rispetto alla richiesta posta. Riesco a comprendere la risposta, facendo un ragionamento deduttivo e non semplicemente attenendomi a ciò che il chatbot mi ha fornito. Potrei aver bisogno di ulteriori chiarimenti.
4. **Risposta molto pertinente:** Il chatbot ha compreso la mia domanda. La risposta fornita rispecchia ciò che volevo richiedere, pur essendo parziale e dimenticando alcuni dettagli della risposta attesa.

5. **Risposta esaustiva:** Il chatbot ha compreso a pieno la mia domanda. La risposta fornita soddisfa completamente la mia richiesta e non ho bisogno di ulteriori chiarimenti.

Valutazione di un esperto

Per ottenere una valutazione più coerente ed esaustiva, oltre al feedback degli utenti, è stato considerato anche il feedback di un esperto dell'architettura del sistema, che in questo caso coincide con l'autore del presente lavoro. L'esperto ha come compito quello di dare un giudizio da 1 a 5 alla risposta fornita dal sistema. Il suo giudizio ha una funzione diversa rispetto a quello dell'utente: mentre il primo valuta in base a quanto le risposte fornite gli aggradano, l'esperto valuta le risposte in base alla correttezza dell'interpretazione dell'input da parte del sistema e all'accuratezza delle risposte generate. In altre parole, l'esperto si occupa di confrontare le risposte date e valutarle in relazione alle migliori risposte ammesse dal chatbot.

La valutazione dell'esperto è espressa tramite la seguente scala:

1. **Il sistema non ha compreso la richiesta e la risposta non è pertinente:** Il chatbot non ha compreso la richiesta rispondendo in modo errato o con un messaggio che chiede all'utente di chiarire la propria richiesta. Il sistema avrebbe potuto rispondere correttamente alla domanda, ma l'input dell'utente o la relativa traduzione non ha intercettato il pattern corretto.
2. **Il sistema non ha fornito una risposta adeguata perché non ha le possibilità di rispondere alla domanda:** Il chatbot non ha compreso la richiesta pertanto ha risposto in modo errato o con un messaggio di chiarimento. Il sistema implementato non ha però possibilità di rispondere, a causa dei limiti dovuti al linguaggio AIML o a causa di un errore nel porre la richiesta da parte dell'utente. Non esiste alcuna risposta predefinita che risponda correttamente alla domanda posta.
3. **Il sistema ha fornito una risposta pertinente, ma parziale:** Il chatbot ha compreso almeno in parte la domanda dell'utente. La risposta fornita non è perfetta e avrebbe potuto essere migliore. Si riesce a comprendere la risposta solamente mediante un ragionamento deduttivo e non semplicemente attenendosi alla risposta fornita dal chatbot.
4. **Il sistema ha fornito una risposta adeguata, ma esiste una risposta più precisa:** Il chatbot ha risposto correttamente alla domanda, ma esiste una risposta predefinita migliore, in termini di contenuti e pertinenza, rispetto alla richiesta in questione.
5. **Il sistema ha risposto al massimo delle sue potenzialità:** Il chatbot ha risposto con la miglior risposta possibile. La risposta è coerente, esaustiva ed inerente al contesto.

Registrazione dei dati

Per garantire una corretta analisi dei dati di valutazione del sistema, tutte le informazioni raccolte tramite l'interfaccia utente vengono memorizzate in un database *MongoDB*¹. Ogni interazione tra l'utente e il chatbot viene salvata come una tupla contenente le informazioni dettagliate.

¹MongoDB è un database management system non relazionale orientato ai documenti.

```
_id: ObjectId('67092b0d9aab917962cd068a')
conversationNumber : 11
questionNumber : 1
input : "give me the number of nodes"
translatedInput : "How many states are there?"
originalInputResponse : "I dont' know. Try asking something like, "Which is the initial state?"
originalInputResponseVote : 2
expertOriginalInputResponseVote : 1
translatedInputResponse : "There are a total of 5 states. q0, q1, q2, q3, and q4. q0 is both init..."
translatedInputResponseVote : 4
expertTranslatedInputResponseVote : 5
createdAt : 2024-10-09T17:16:24.459+00:00
updatedAt : 2024-10-09T17:19:27.843+00:00
__v : 0
```

Figura 4.3. Struttura di una tupla rappresentante una conversazione tra un utente e il chatbot. Ogni attributo della tupla registra un aspetto specifico dell'interazione, compresi gli input originali e tradotti, le risposte del chatbot e le valutazioni.

Nello specifico, ogni tupla contiene i seguenti attributi:

- **_id**: Identificatore univoco dell'oggetto, utilizzato per identificare la tupla nel database.
- **conversationNumber**: Numero che identifica la conversazione specifica all'interno del set di dati.
- **questionNumber**: Numero della domanda all'interno della conversazione.
- **input**: L'input originale fornito dall'utente al chatbot.
- **translatedInput**: La versione tradotta dell'input dell'utente, utilizzata per migliorare la comprensione del modello.
- **originalInputResponse**: La risposta generata dal chatbot in risposta all'input originale non tradotto.
- **originalInputResponseVote**: Voto assegnato dall'utente per la qualità della risposta generata all'input originale.
- **expertOriginalInputResponseVote**: Valutazione data dall'esperto riguardo alla risposta all'input originale.
- **translatedInputResponse**: La risposta del chatbot in base all'input tradotto.
- **translatedInputResponseVote**: Voto assegnato dall'utente per la qualità della risposta generata all'input tradotto.
- **expertTranslatedInputResponseVote**: Valutazione data dall'esperto riguardo alla risposta all'input tradotto.
- **createdAt**: Data e ora in cui la tupla è stata creata.
- **updatedAt**: Data e ora dell'ultimo aggiornamento della tupla.
- **__v**: Versione del documento, utilizzata da MongoDB per il controllo delle versioni.

Questi dati sono stati registrati automaticamente durante ogni interazione e aggiornati in tempo reale. L'utilizzo di MongoDB consente la gestione dei dati in modo che possano essere facilmente utilizzati per eseguire analisi statistiche sulle performance del sistema. I dati raccolti e il codice delle analisi condotte sono disponibili nella cartella `final_evaluation` della repository GitHub menzionata nella Sezione 3.1.

Selezione dei candidati e gestione della valutazione

Per garantire una valutazione accurata del sistema, è stato necessario definire criteri precisi per la selezione dei candidati. In totale, sono stati coinvolti 10 partecipanti a base volontaria, di cui 8 uomini e 2 donne, tutti con una buona conoscenza degli automi a stati finiti e della lingua inglese. La maggior parte dei candidati erano studenti universitari in ambito informatico o ingegneristico, con un'età compresa tra i 22 e i 27 anni (età media intorno ai 24 anni). In futuro, potrebbe essere utile espandere la fase di valutazione a partecipanti di età superiore e con differenti livelli di scolarizzazione, al fine di esplorare se esistano differenze nella percezione delle risposte in funzione dell'età e del background educativo.

Durante la fase di valutazione, a ciascun partecipante è stata fornita una copia stampata della scala di valutazione con la relativa spiegazione. Non sono state date ulteriori istruzioni, se non quella di cercare di ottenere quante più informazioni possibili, riguardanti l'automa a stati finiti descritto dal chatbot, e di valutare immediatamente la pertinenza delle risposte fornite. La valutazione è stata condotta in un ambiente isolato: ogni partecipante si trovava in una stanza separata dagli altri per evitare condizionamenti o scambio di informazioni implicite, per prevenire possibili bias o pregiudizi. Nessun partecipante è stato informato anticipatamente sullo scopo specifico della valutazione, in modo da ottenere risultati il più possibile oggettivi e non influenzati. Inoltre, non è stato posto alcun limite al numero di domande che i partecipanti potevano porre al chatbot durante la conversazione, offrendo loro maggiore libertà e permettendo di raccogliere un ulteriore dato da analizzare.

4.2 Analisi dei risultati della valutazione

In questa sezione vengono analizzati i risultati della valutazione del sistema in termini di voti assegnati dagli utenti e dagli esperti per le risposte generate dal chatbot. L'analisi si focalizza sul confronto tra le risposte fornite dal sistema in risposta agli input originali e quelle generate in seguito alla traduzione degli input in linguaggio naturale controllato.

I dati raccolti e il codice delle analisi condotte sono disponibili nella cartella `final_evaluation` della repository GitHub menzionata nella Sezione 3.1.

Confronto delle valutazioni registrate

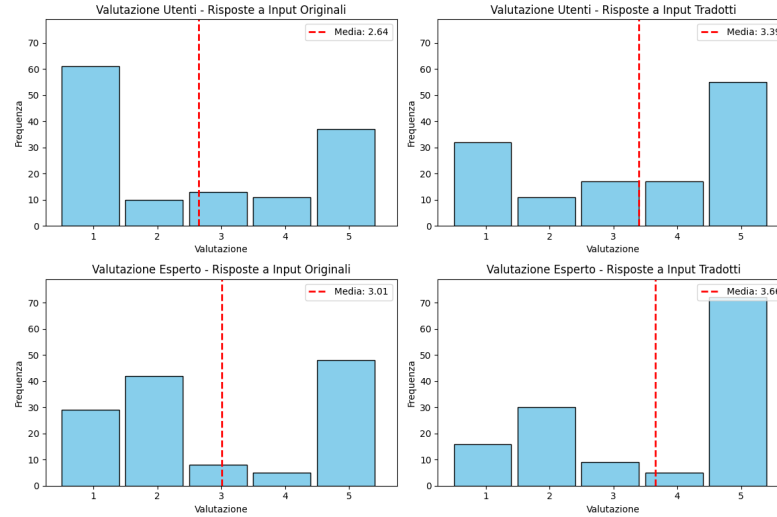


Figura 4.4. Analisi comparativa delle votazioni degli utenti e dell'esperto per le risposte ad input originali e tradotti. I grafici illustrano la distribuzione dei voti ricevuti per le risposte agli input originali (a sinistra) e per le risposte agli input tradotti (a destra), sia dal punto di vista degli utenti che dell'esperto. In ogni grafico, la linea tratteggiata rossa rappresenta il valore medio della votazione.

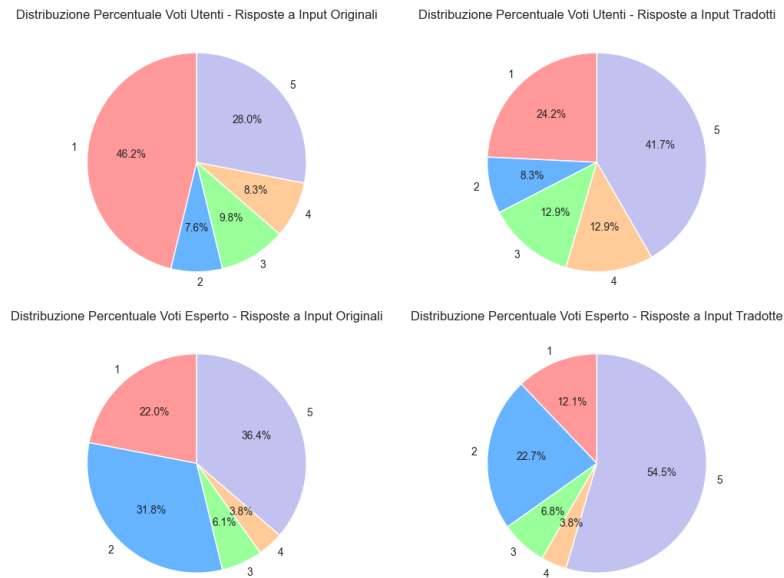


Figura 4.5. Distribuzione percentuale dei voti assegnati dagli utenti e dall'esperto per le risposte generate in base agli input originali e tradotti.

I grafici 4.4 e 4.5 mostrano in due modi diversi le valutazioni registrate dagli utenti e dall'esperto per le risposte generate dal chatbot. Le due analisi evidenziano come i voti assegnati dagli utenti, alle risposte derivanti da input originali, mostrano un picco rilevante sul voto 1, con una media di 2.64. Questo indica che una parte significativa delle risposte non è stata considerata pertinente o utile dagli utenti. Un andamento simile si osserva nelle valutazioni dell'esperto, con una media di 3.01 per le risposte agli input originali. D'altra parte, i voti assegnati alle risposte generate a partire dagli input tradotti, mostrano una distribuzione più favorevole. Gli utenti, in questo caso, hanno assegnato voti più alti, con un picco sul voto 5 e una media di 3.39. Questo miglioramento indica che la traduzione dell'input dell'utente ha avuto un effetto positivo sulla qualità percepita delle risposte generate dal chatbot. La stessa tendenza è confermata anche dai voti assegnati dall'esperto, con una media di 3.66 per le risposte agli input tradotti, superiore alla media delle risposte agli input originali.

I grafici 4.5 rafforzano questa analisi mostrando la distribuzione percentuale dei voti. Per gli utenti, il voto 1 è stato assegnato al 46.2% delle risposte basate su input originali. Tale percentuale, dopo la traduzione, è scesa al 24.2%. Allo stesso modo, la percentuale di voti 5 è aumentata dal 28.0% al 41.7% per gli input tradotti. Anche la valutazione dell'esperto evidenzia un cambiamento analogo: il voto 5 è passato dal 36.4%, per le risposte basate su input originali, al 54.5%, per quelle basate su input tradotti. Mentre, il voto 1, è sceso dal 22.0% al 12.1%.

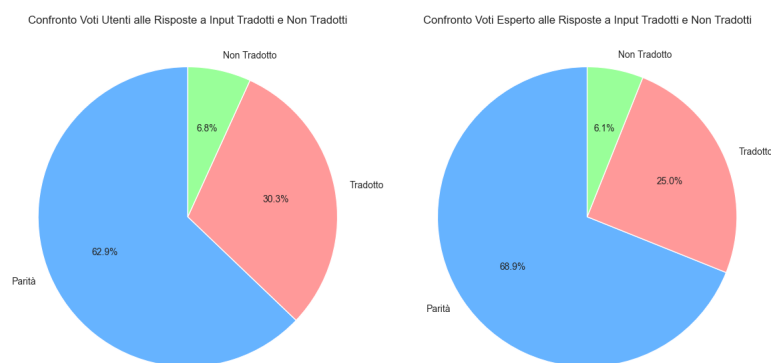


Figura 4.6. Confronto delle percentuali di voti assegnati dagli utenti e dall'esperto alle risposte generate in base agli input tradotti e non tradotti.

I due grafici a torta mostrano il confronto tra i voti assegnati dagli utenti e dall'esperto per le risposte del chatbot basate su input tradotti e non tradotti. Per quanto riguarda gli utenti, il 30.3% delle risposte agli input tradotti ha ottenuto voti migliori rispetto a quelle di input non tradotti. Il 62.9% delle risposte è stato giudicato equivalente, ossia hanno ottenuto la stessa valutazione. Solo il 6.8% delle risposte ad input non tradotti ha ricevuto voti superiori. Un pattern simile si osserva per le valutazioni dell'esperto: una maggioranza del 68.9% di voti in parità e una significativa percentuale in cui le risposte agli input tradotti hanno ottenuto voti migliori. Questi risultati confermano che la traduzione degli input ha, generalmente, migliorato la qualità percepita delle risposte, sia dagli utenti che dall'esperto.

Analisi delle conversazioni e della differenza di valutazione fra utenti e esperto

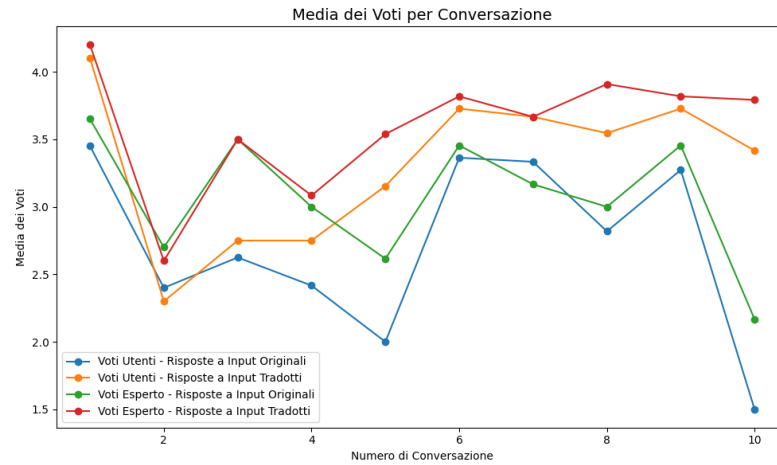


Figura 4.7. Andamento della media dei voti assegnati dagli utenti e dall'esperto, per le risposte generate dal chatbot nelle diverse conversazioni. Il grafico confronta le valutazioni delle risposte basate su input originali e tradotti, sia per gli utenti che per l'esperto. Mostra come variano le medie dei voti nelle diverse conversazioni.

Il grafico 4.7 rappresenta l'andamento della media dei voti assegnati dagli utenti e dall'esperto per le risposte del chatbot nelle diverse conversazioni, distinguendo tra risposte generate a partire da input originali e risposte basate su input tradotti.

Dai dati emerge che le risposte basate su input tradotti (linee arancione e rossa) tendono a ottenere voti medi più elevati rispetto alle risposte basate su input originali (linee blu e verde). Questo avviene sia per quanto riguarda le valutazioni degli utenti che quelle dell'esperto. In particolare, le risposte tradotte mantengono una media dei voti più stabile e alta, con valori che si attestano intorno al 3.5-4 per buona parte delle conversazioni. Invece, le risposte basate su input originali, al contrario, mostrano maggiore variabilità. Le valutazioni degli utenti (linea blu) tendono a essere più basse, con un trend decrescente che si intensifica nelle ultime conversazioni.

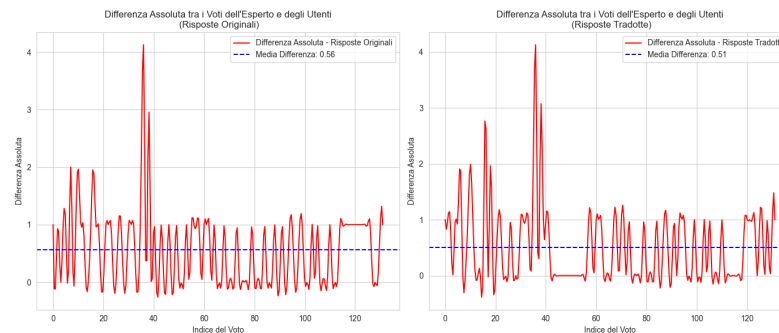


Figura 4.8. Differenza assoluta tra i voti dell'esperto e quelli degli utenti, per le risposte generate in base agli input originali (a sinistra) e agli input tradotti (a destra). La linea rossa rappresenta la differenza assoluta tra i voti assegnati per ogni singola risposta, mentre la linea blu tratteggiata indica la media della differenza assoluta su tutte le risposte.

I due grafici mostrano la differenza assoluta tra i voti assegnati dagli utenti e dall'esperto per le risposte generate dal chatbot, distinguendo tra input originali (grafico a sinistra) e input tradotti (grafico a destra). La differenza assoluta rappresenta la discrepanza tra il voto dell'utente e quello dell'esperto per ogni singola risposta.

Nel grafico a sinistra, relativo agli input originali, è possibile osservare un andamento irregolare, con picchi significativi di differenza in alcune risposte, dove la discrepanza tra il giudizio dell'esperto e quello degli utenti è più marcata. La media della differenza assoluta per le risposte originali è di 0.56, come indicato dalla linea blu tratteggiata. Nel grafico a destra, relativo agli input tradotti, il comportamento è simile ma con una media leggermente inferiore, ossia 0.51. Anche qui si possono osservare picchi di differenza, anche se le oscillazioni appaiono leggermente più contenute rispetto alle risposte originali. Questo suggerisce che la traduzione degli input ha contribuito a ridurre, seppur in modo limitato, la discrepanza tra le valutazioni degli utenti e dell'esperto.

Analisi dei casi in cui interviene il processo di traduzione

In questa sezione, ci concentriamo sui casi in cui il processo di traduzione interviene per trasformare l'input originale dell'utente in linguaggio naturale controllato. Quando l'input originale è già ben formato, intercettando direttamente un pattern AIML, non è necessario alcun processo di traduzione e, di conseguenza, gli attributi `input` e `translatedInput` risultano uguali. Tuttavia, quando l'input dell'utente non riesce a intercettare un pattern utile, il sistema applica la traduzione in CNL per facilitare la risposta del chatbot, portando a differenze tra i campi `input` e `translatedInput`.

Distribuzione Percentuale delle Differenze tra Input e Input Tradotto

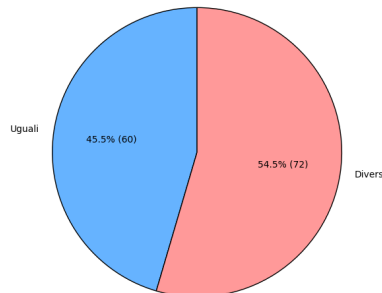


Figura 4.9. Distribuzione percentuale delle differenze tra i campi `input` e `translatedInput`. Il grafico a torta mostra la proporzione di tuple in cui l'input originale (`input`) è identico all'input tradotto (`translatedInput`) rispetto a quelle in cui i due valori differiscono.

Il grafico 4.9 mostra la distribuzione percentuale delle tuple in cui l'`input` e il `translatedInput` sono uguali o diversi. Il 54.5% delle tuple presenta input diversi, indicando che il processo di traduzione è stato necessario per migliorare la formulazione dell'input e intercettare un pattern AIML utile. Al contrario, nel 45.5% dei casi l'input era già ben formato, intercettando direttamente un pattern AIML senza la necessità di ulteriori trasformazioni tramite traduzione. Dunque, poco più di un input su due richiede un intervento del processo di traduzione per generare una risposta pertinente, evidenziando l'importanza di questa fase nel miglioramento dell'efficacia del chatbot.

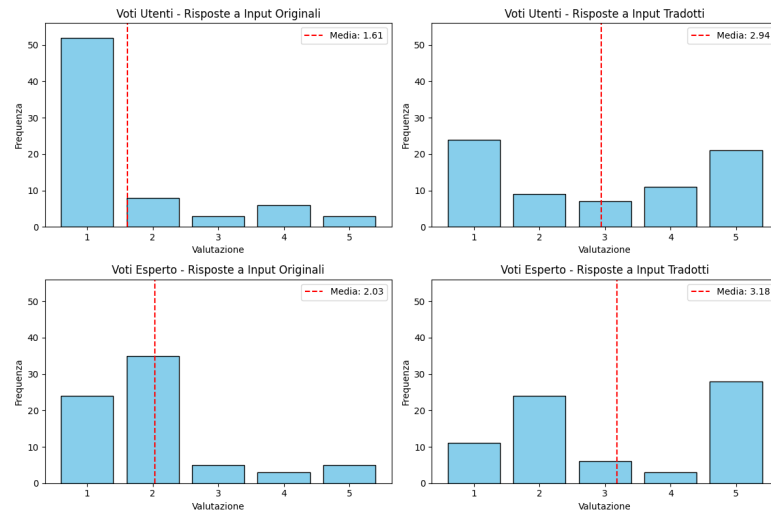


Figura 4.10. Distribuzione dei voti assegnati dagli utenti e dall'esperto per le risposte generate a partire da input originali e tradotti. Il grafico rappresenta esclusivamente le valutazioni relative alle tuple in cui **input** e **translatedInput** differiscono, ovvero nei casi in cui è stato applicato il processo di traduzione.

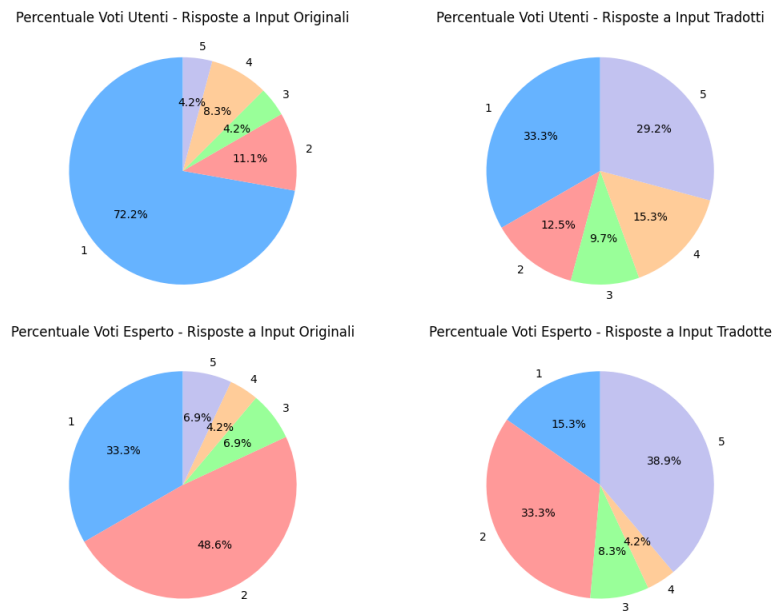


Figura 4.11. Distribuzione percentuale dei voti assegnati dagli utenti e dall'esperto per le risposte generate a partire da input originali e tradotti. Il grafico rappresenta esclusivamente le valutazioni relative alle tuple in cui **input** e **translatedInput** differiscono, ovvero nei casi in cui è stato applicato il processo di traduzione.

I grafici presentati mostrano l'analisi delle valutazioni assegnate dagli utenti e dall'esperto, esclusivamente per le risposte generate a partire da input che hanno fatto scattare il processo di traduzione, ossia nei casi in cui `input` e `translatedInput` sono risultati diversi. La figura 4.10 presenta istogrammi con la frequenza delle valutazioni e la media rappresentata da una linea verticale rossa tratteggiata, mentre il secondo grafico utilizza diagrammi a torta per mostrare la distribuzione percentuale delle votazioni.

Nell'analisi delle valutazioni degli utenti, emerge che, per le risposte agli input originali, il 72.2% dei voti è 1, segnalando una forte insoddisfazione per queste risposte. Tuttavia, per le risposte generate dagli input tradotti, solo il 33.3% dei voti è 1. D'altro canto la percentuale di voti 5 aumenta significativamente, dal 4.2% al 29.2%. Questo mostra che il processo di traduzione ha portato a una distribuzione delle valutazioni più equilibrata, con una media che passa da 1.61 per le risposte agli input originali a 2.94 per le risposte agli input tradotti. Per quanto riguarda le valutazioni dell'esperto, il pattern è simile: il 48.6% dei voti per le risposte agli input originali è 2, con una media di 2.03. D'altra parte, per le risposte generate dagli input tradotti, la media sale a 3.18, con il 38.9% delle valutazioni che raggiungono il voto massimo di 5.

Nonostante l'intervento del processo di traduzione, è evidente che in alcuni casi le valutazioni rimangono basse. Questo fenomeno è comprensibile poiché, anche se la traduzione migliora la formulazione dell'input, il chatbot basato su AIML dispone di un set finito di risposte. Ciò significa che, in alcune situazioni, il sistema potrebbe non essere in grado di fornire una risposta soddisfacente se il costruttore del chatbot non ha considerato tutte le possibili varianti o sfaccettature della conversazione. Di conseguenza, anche dopo il processo di traduzione, possono verificarsi situazioni in cui la risposta non corrisponde alle aspettative o ai bisogni dell'utente.

Capitolo 5

Conclusioni, risultati ottenuti e possibili sviluppi futuri

In questa tesi è stato affrontato il problema della traduzione dell'input utente per migliorare l'interazione con il chatbot NoVAGraphS, basato su regole AIML. Attraverso l'impiego di modelli di linguaggio di grandi dimensioni e tecniche di few-shot prompting, è stata proposta un'architettura capace di tradurre l'input, riducendo le ambiguità e migliorando la qualità delle risposte generate.

Il sistema proposto ha dimostrato un miglioramento nella gestione degli input, soprattutto nei casi in cui l'input non corrisponde direttamente ad un pattern AIML predefinito. L'adozione di tecniche di embedding semantico, per la verifica della pertinenza, ha permesso di identificare correttamente i pattern AIML più rilevanti, mentre il modello LLM ha gestito la traduzione dell'input non riconosciuto. Questo ha portato a una riduzione significativa delle risposte errate o non pertinenti, migliorando la qualità complessiva delle interazioni.

Nonostante i risultati positivi, il sistema presenta alcune limitazioni che meritano di essere discusse. In primo luogo, sebbene il modello LLM sia stato efficace nella maggior parte dei casi, sono stati evidenziati problemi legati alla generazione di risposte non accurate. Un'evoluzione del sistema potrebbe consistere nell'integrazione di modelli LLM più avanzati, capaci di gestire meglio le complessità del linguaggio naturale, riducendo ulteriormente i casi di allucinazione e migliorando la pertinenza delle risposte.

Un'altra limitazione risiede nell'incapacità di gestire input completamente fuori contesto: sebbene il sistema sia in grado di tradurre input ambigui o parzialmente corretti, in alcuni casi, non è stato possibile trovare una corrispondenza adeguata nei pattern AIML, portando a risposte errate o incomplete. Questo è dovuto alla natura deterministica del sistema AIML, che non consente al chatbot di adattarsi agli input imprevisti senza una regola predefinita. Una possibile evoluzione è l'aggiunta di nuove regole AIML, basate sugli input che generano risposte non pertinenti. Questo approccio potrebbe rendere il chatbot più flessibile nella gestione di input non attualmente previsti, ampliando così le possibilità di conversazione del sistema e migliorando la qualità delle risposte.

Infine, l'utilizzo del few-shot prompting ha mostrato dei limiti di scalabilità. Sebbene efficace in contesti ristretti, questa tecnica potrebbe non essere sufficiente per gestire input che si discostano significativamente dagli esempi forniti. In tali situazioni, sarebbe necessario incrementare il numero di esempi o adottare tecniche di fine-tuning del modello, per adattarlo meglio al dominio specifico del chatbot. Il fine-tuning è il processo attraverso cui un modello di intelligenza artificiale pre-addestrato viene adattato utilizzando un nuovo set di dati specifico, per migliorare le sue

prestazioni su compiti specifici e in domini particolari. Infatti, attualmente, il modello utilizzato non è specificamente addestrato sul dominio degli automi a stati finiti, limitando la comprensione dei dettagli tecnici nelle richieste degli utenti.

In conclusione, questa tesi ha messo in luce come l'integrazione di tecnologie avanzate, possa migliorare significativamente le interazioni tra utenti e chatbot basati su AIML. I risultati ottenuti indicano che, con ulteriori sviluppi, è possibile realizzare chatbot sempre più efficaci, capaci di gestire input complessi e fornire risposte pertinenti e accurate.

Bibliografia

- [1] Giorgio Ausiello, Fabrizio d'Amore e Giorgio Gambosi. *Linguaggi, Modelli, Complessità*. Franco Angeli Editore, 2003, pp. 71–82.
- [2] Pier Felice Balestrucci, Elisa Di Nuovo, Manuela Sanguinetti, Luca Anselma, Cristian Bernareggi e Alessandro Mazzei. *Designing a Dialogue System for Making Finite State Automata Accessible*. Preprint, Submitted. Submitted for publication. 2023.
- [3] Daniel G. Bobrow, Ronald M. Kaplan, Martin Kay, Donald A. Norman, Henry Thompson e Terry Winograd. “GUS, A Frame-Driven Dialog System”. In: *Artificial Intelligence* 8 (1977), pp. 155–173.
- [4] Tim Bray, Jean Paoli, C. M. Sperberg-McQueen, Eve Maler, François Yergeau e John Cowan. *Extensible Markup Language (XML) 1.1 (Second Edition)*. W3C Recommendation. 2006. URL: <http://www.w3.org/TR/2006/REC-xml11-20060816>.
- [5] DeepLearning.AI. *ChatGPT Prompt Engineering for Developers*. <https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/>. Accesso il 13 marzo 2024. 2024.
- [6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini e Hervé Jégou. “The Faiss Library”. In: *arXiv preprint arXiv:2401.08281* (2024). URL: <https://arxiv.org/abs/2401.08281>.
- [7] Jeffrey E. F. Friedl. *Mastering Regular Expressions*. 3rd. O'Reilly Media, 2006.
- [8] Osservatori Digital Innovation. *Cos'è il Natural Language Processing (NLP) e come funziona*. https://blog.osservatori.net/it_it/natural-language-processing-nlp-come-funziona-lelaborazione-del-linguaggio-naturale. Accesso il 2 agosto 2024. 2024.
- [9] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto e Pascale Fung. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023). March 2023, Article 248.
- [10] Zhi Jing, Yongye Su, Yikun Han, Bo Yuan, Haiyun Xu, Chunjiang Liu, Kehai Chen e Min Zhang. “When Large Language Models Meet Vector Databases: A Survey”. In: *arXiv preprint arXiv:2402.01763* (2024). URL: <https://arxiv.org/abs/2402.01763>.
- [11] Daniel Jurafsky. *CS124: From Languages to Information*. <https://web.stanford.edu/class/cs124/>. Accesso il 25 ottobre 2023. 2023.
- [12] Daniel Jurafsky e James H. Martin. *Speech and Language Processing*. Bozza non pubblicata, Capitolo 7: Neural Networks and Neural Language Models, versione del 25 ottobre 2021. 2021.
- [13] Daniel Jurafsky e James H. Martin. *Speech and Language Processing*. Bozza non pubblicata, Capitolo 15: Chatbots and Dialogue Systems, versione del 7 gennaio 2023. 2023.

- [14] LangChain. *Vector Stores Documentation*. https://python.langchain.com/v0.1/docs/modules/data_connection/vectorstores/. Accesso il 20 marzo 2024. 2024.
- [15] J.C.M. Mallery. *Thinking About Foreign Policy: Finding an Appropriate Role for Artificially Intelligent Computers*. The 1988 Annual Meeting of the International Studies Association, St. Louis, Missouri. 1988.
- [16] Christopher D. Manning, Prabhakar Raghavan e Hinrich Schütze. *Introduction to Information Retrieval*. Consultato Capitolo 6. Cambridge University Press, 2008.
- [17] ngrok. *ngrok Documentation*. <https://ngrok.com/docs>. Accesso il 15 giugno 2024. 2024.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015, pp. 1–15.
- [19] Elisa Di Nuovo, Manuela Sanguinetti, Pier Felice Balestrucci, Luca Anselma, Cristian Bernareggi e Alessandro Mazzei. “Educational Dialogue Systems for Visually Impaired Students: Introducing a Task-Oriented User-Agent Corpus”. In: *Proceedings of the LREC-COLING 2024*. CC BY-NC 4.0. 20-25 May, 2024: ELRA Language Resource Association, 2024, pp. 5507–5519.
- [20] John D. Owens, Mike Houston, David Luebke, Simon Green, John E. Stone e James C. Phillips. “GPU Computing”. In: *Proceedings of the IEEE* 96.5 (2008), pp. 879–899.
- [21] Pandorabots. *AIML Reference Documentation*. <https://www.pandorabots.com/docs/aiml-reference/>. Accesso il 10 novembre 2023. 2023.
- [22] Google Research. *Google Colaboratory FAQ*. <https://research.google.com/colaboratory/faq.html>. Accesso il 20 marzo 2024. 2024.
- [23] Edoardo Rinaldi. “Q&A Information Retrieval: Progettazione, osservazione ed analisi di una sperimentazione didattica sull’intelligenza artificiale generativa nell’ambito di un corso di Basi di Dati”. Consultato fino a pagina 13. Politecnico di Torino, 2024.
- [24] Giovanni Siragusa. *Prompt Engineering*. Gruppo Social Computing, Dipartimento di Informatica, UniTO. 2024.
- [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser e Illia Polosukhin. “Attention is All You Need”. In: *Neural Information Processing Systems (NeurIPS)*. 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [26] Richard S. Wallace. “The Anatomy of A.L.I.C.E.” In: *Parsing the Turing Test*. A cura di Robert Epstein, Gary Roberts e Grace Beber. Springer Science, 2009.
- [27] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le e Denny Zhou. “Chain of Thought Prompting Elicits Reasoning in Large Language Models”. In: *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS 2022)*. 2022. URL: <https://arxiv.org/abs/2201.11903>.
- [28] Joseph Weizenbaum. “ELIZA—A Computer Program For the Study of Natural Language Communication Between Man and Machine”. In: *CACM* 9.1 (1966), pp. 36–45.